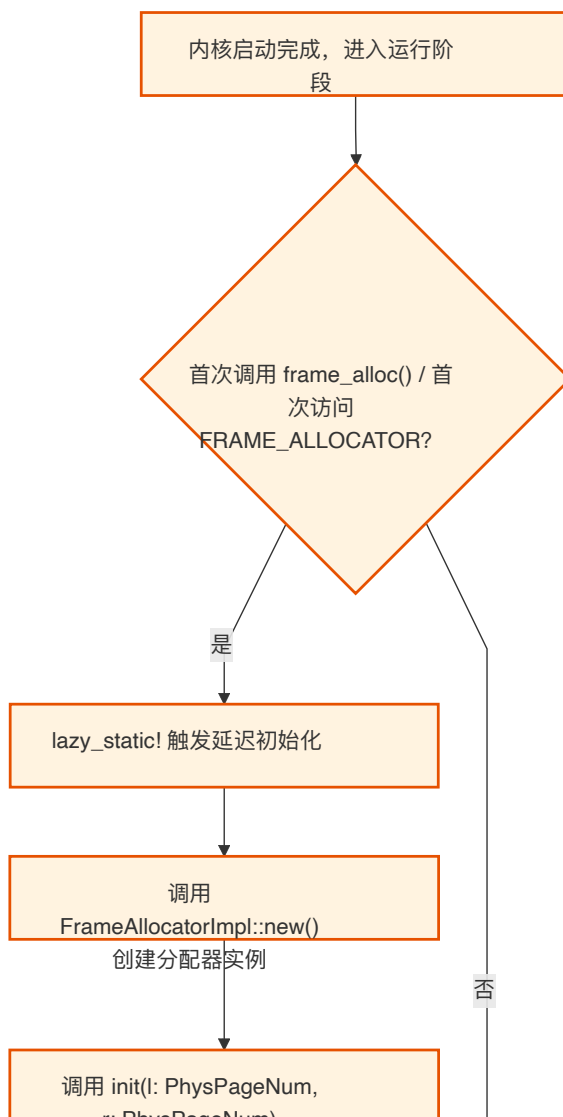


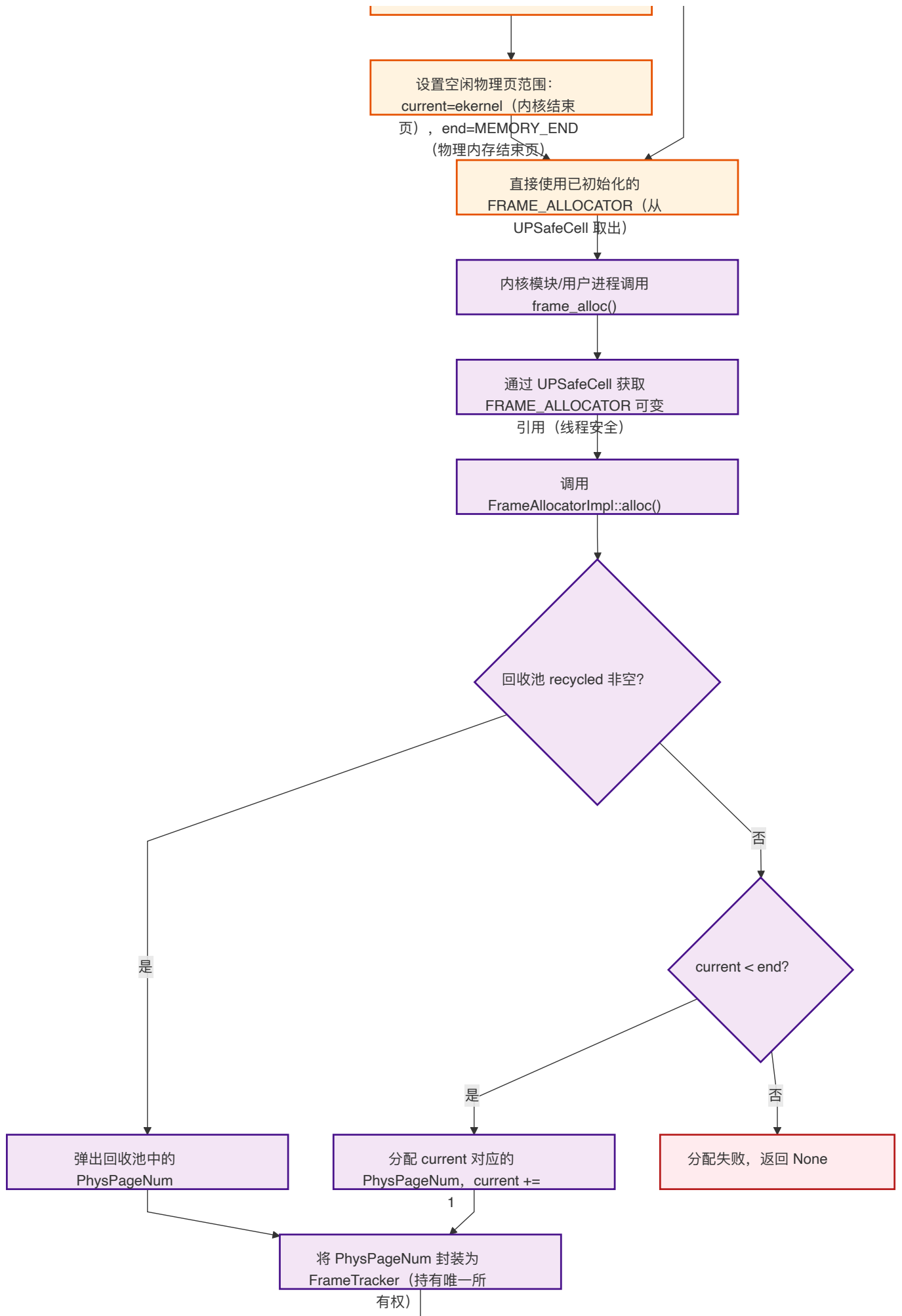
在这个章节，我们会实现虚拟内存机制。

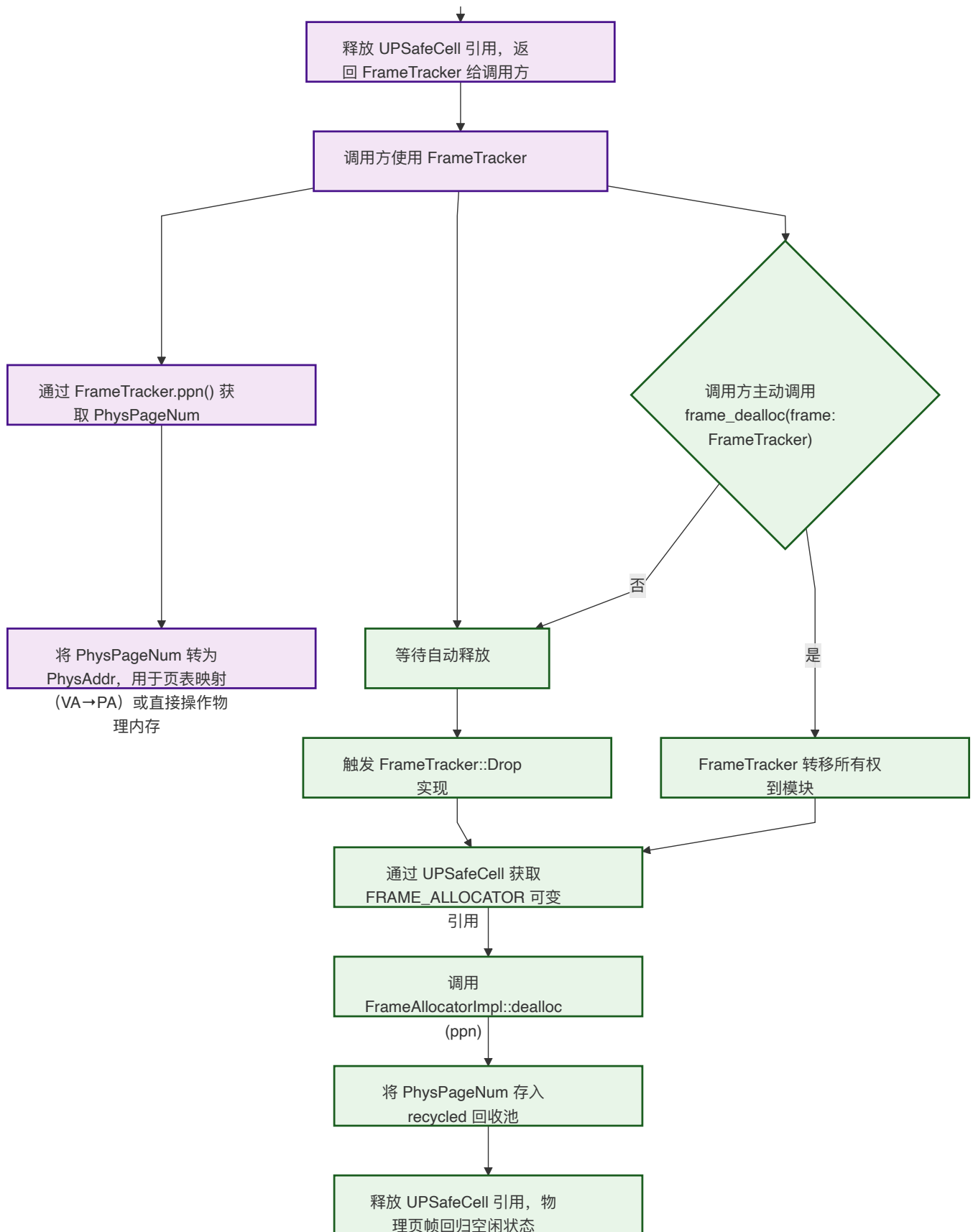
```
1 → ~/code/2025a-rcore-0x822a5b87 git:(ch4) cloc --include-ext=rs,s,S,asm os
2     152 text files.
3     121 unique files.
4     126 files ignored.
5
6 github.com/AlDanial/cloc v 1.82  T=0.01 s (2328.7 files/s, 184956.7 lines/s)
7 -----
8 Language                      files          blank          comment          code
9 -----
10 Rust                          29            193            386            1866
11 Assembly                      4             10             26             140
12 -----
13 SUM:                          33            203            412            2006
14 -----
```

管理SV39多级页表

物理页帧管理





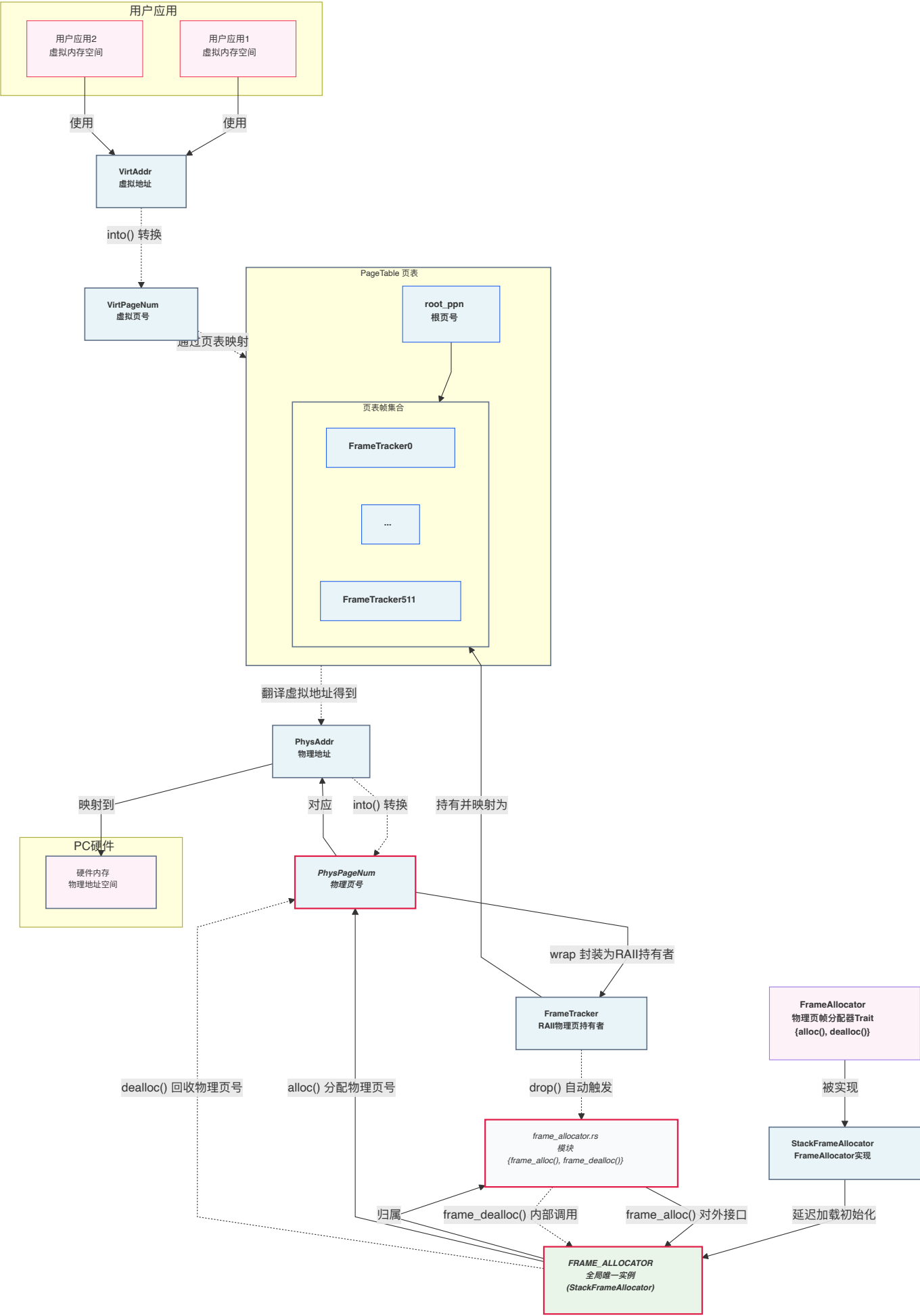


物理页帧存放的数据从内容上来说分为两种：

1. 应用/内核的数据或者代码；
2. 应用/内核的多级页表；

本质上，物理页帧的核心是“存储单元”—— 无论存放的是代码、应用数据，还是页表本身，在物理内存层面都是“二进制数据块”，页表只是“具有特殊用途的数据”（用于地址转换的“映射表数据”）

可用物理页的分配与回收



区分内核空间和物理空间可以参考 [内存的内核空间 and 用户空间](#)

物理页的实际大小，由三个参数共同决定。他们大小从也是 `virt memory < ekernel < MEMORY_END`。这里需要注意的是，这三个值都是物理地址而非虚拟地址。

1. qemu模拟的 `virt` 硬件平台上的物理内存的起始地址 `0x80000000`；
2. `ekernel` 在 `linker` 中，我们在完成 `.bss` 并且进行内存对齐之后，指定了 `ekernel` 这个参数，表示内核代码的结束位置；
3. `MEMORY_END` 在 `config.rs` 中硬编码的地址 `0x88000000`。

随后，我们定义：

1. `trait FrameAllocator` 用于管理我们全部可用的物理内存；
2. `impl StackFrameAllocator` 是基于Vec的一个简单实现；

```
1 // os/src/mm/frame_allocator.rs
2
3 trait FrameAllocator {
4     fn new() -> Self;
5     fn alloc(&mut self) -> Option<PhysPageNum>;
6     fn dealloc(&mut self, ppn: PhysPageNum);
7 }
```

`StackFrameAllocator` 的实现逻辑也非常简单：

1. `current` 和 `end` 表示空闲内存的页号，这个页号初始由 `ekernel` 和 `MEMORY_END` 进行内存对齐后计算得出；
2. `recycled` 表示 `alloc` 后 `dealloc` 的内存，我们分配时也优先从这里开始分配；

```
1 // os/src/mm/frame_allocator.rs
2
3 pub struct StackFrameAllocator {
4     current: usize, //空闲内存的起始物理页号
5     end: usize,     //空闲内存的结束物理页号
6     recycled: Vec<usize>,
7 }
8
9 // os/src/mm/frame_allocator.rs
10
11 impl FrameAllocator for StackFrameAllocator {
12     fn new() -> Self {
13         Self {
14             current: 0,
15             end: 0,
16             recycled: Vec::new(),
17         }
18     }
19     fn alloc(&mut self) -> Option<PhysPageNum> {
20         if let Some(ppn) = self.recycled.pop() {
21             Some(ppn.into())
22         }
23     }
24 }
```

```

22     } else {
23         if self.current == self.end {
24             None
25         } else {
26             self.current += 1;
27             Some((self.current - 1).into())
28         }
29     }
30 }
31 fn dealloc(&mut self, ppn: PhysPageNum) {
32     let ppn = ppn.0;
33     // validity check
34     if ppn >= self.current || self.recycled
35         .iter()
36         .find(|&v| {*v == ppn})
37         .is_some() {
38         panic!("Frame ppn={:#x} has not been allocated!", ppn);
39     }
40     // recycle
41     self.recycled.push(ppn);
42 }
43 }
44
45 impl StackFrameAllocator {
46     pub fn init(&mut self, l: PhysPageNum, r: PhysPageNum) {
47         self.current = l.0;
48         self.end = r.0;
49     }
50 }

```

随后初始化一个全局的 `FRAME_ALLOCATOR`，这里需要注意的是，我们要把物理地址 `ekernel` 和 `MEMORY_END` 转换为对应的页号。

```

1  type FrameAllocatorImpl = StackFrameAllocator;
2
3  lazy_static! {
4      /// frame allocator instance through lazy_static!
5      pub static ref FRAME_ALLOCATOR: UPSafeCell<FrameAllocatorImpl> =
6          unsafe { UPSafeCell::new(FrameAllocatorImpl::new()) };
7  }
8  /// initiate the frame allocator using `ekernel` and `MEMORY_END`
9  pub fn init_frame_allocator() {
10     extern "C" {
11         fn ekernel();
12     }
13     FRAME_ALLOCATOR.exclusive_access().init(
14         PhysAddr::from(ekernel as usize).ceil(),
15         PhysAddr::from(MEMORY_END).floor(),
16     );
17 }

```

分配/回收物理页帧的接口

1. `alloc` 和 `dealloc` 都是非pub的函数，我们需要对外暴露分配的接口。
2. 这里非常值得注意的一点是，我们 `frame_alloc()` 分配得到的是 `FrameTracker` 而不是 `PhysPageNum`，这个设计很巧妙，可以参考 [PhysPageNum和FrameTracker](#)

```
1 // os/src/mm/frame_allocator.rs
2
3 pub fn frame_alloc() -> Option<FrameTracker> {
4     FRAME_ALLOCATOR
5         .exclusive_access()
6         .alloc()
7         .map(|ppn| FrameTracker::new(ppn))
8 }
9
10 fn frame_dealloc(ppn: PhysPageNum) {
11     FRAME_ALLOCATOR
12         .exclusive_access()
13         .dealloc(ppn);
14 }
```

`FrameTracker` 的实现

```
1 /// tracker for physical page frame allocation and deallocation
2 pub struct FrameTracker {
3     /// physical page number
4     pub ppn: PhysPageNum,
5 }
6
7 impl FrameTracker {
8     /// Create a new FrameTracker
9     pub fn new(ppn: PhysPageNum) -> Self {
10         // page cleaning
11         let bytes_array = ppn.get_bytes_array();
12         for i in bytes_array {
13             *i = 0;
14         }
15         Self { ppn }
16     }
17 }
18
19 impl Drop for FrameTracker {
20     fn drop(&mut self) {
21         frame_dealloc(self.ppn);
22     }
23 }
```

多级页表管理

页表基本数据结构与访问接口

页表是一个多级的数据结构，他的作用就是将 `VPN` 映射到 `PPN`，而这里有一点需要注意的是：

1. 页表本身也是数据，每一个页表都会有一个唯一的入口地址，而这个地址是他的 `PPN`；
2. 页表内部 `frames` 是通过 `FrameTracker` 来实现 `PPN` 管理，这里在我们当前的逻辑下是可以的：因为我们暂时没有共享物理页的场景。在实际的应用中：当多个进程共享一个 `PPN` 时（例如进程创建时的 `COW`），会导致内存管理异常，因为当共享的某一个进程结束时，这个内存将被释放，而另外一个引用该 `PPN` 的进程会引用一个悬垂指针。
3. 此外，在开启分页后，无论是内核态还是用户态都是通过虚拟地址来访问物理内存。也就是说，我们必须将物理内存中所有需要访问的物理页都设定一个 `VPN` -> `PPN` 的映射关系并且存储到页表中。而具体的这个映射逻辑，请参考 [虚拟内存到物理内存](#)。

```
1  /// page table structure
2  pub struct PageTable {
3      /// Each user-facing application has a corresponding unique multi-level page
   table,
4      /// which means the starting address of each page table is unique.
5      /// That also means we must save root ppn as a `identifier`.
6      root_ppn: PhysPageNum,
7      /// `frames` contains all page table entries (including the root PTE) and stores
   them as a vector.
8      frames: Vec<FrameTracker>,
9  }
10
11 impl PageTable {
12     /// set the map between virtual page number and physical page number
13     pub fn map(&mut self, vpn: VirtPageNum, ppn: PhysPageNum, flags: PTEFlags){}
14     /// remove the map between virtual page number and physical page number
15     pub fn unmap(&mut self, vpn: VirtPageNum){}
16 }
```

内核中访问物理页帧的方法

- `get_pte_array` 返回的是一个页表项定长数组的可变引用，代表多级页表中的一个节点；
- `get_bytes_array` 返回的是一个字节数组的可变引用，可以以字节为粒度对物理页帧上的数据进行访问，前面进行数据清零就用到了这个方法；
- `get_mut` 是个泛型函数，可以获取一个恰好放在一个物理页帧开头的类型为 `T` 的数据的可变引用。例如，我们获取 `TrapContext` 就可以使用它。

这里值得注意的是，`get_pte_array` 这个方法只能对于 `PageTable#root_ppn` 调用。

```
1  impl PhysAddr {
2      ///Get mutable reference to `PhysAddr` value
3      /// Get the mutable reference of physical address
4      pub fn get_mut<T>(&self) -> &'static mut T {
5          unsafe { (self.0 as *mut T).as_mut().unwrap() }
6      }
7  }
```

```

8  impl PhysPageNum {
9      /// Get the reference of page table(array of PTEs)
10     pub fn get_pte_array(&self) -> &'static mut [PageTableEntry] {
11         let pa: PhysAddr = (*self).into();
12         unsafe { core::slice::from_raw_parts_mut(pa.0 as *mut PageTableEntry, 512) }
13     }
14     /// Get the reference of page(array of bytes)
15     pub fn get_bytes_array(&self) -> &'static mut [u8] {
16         let pa: PhysAddr = (*self).into();
17         unsafe { core::slice::from_raw_parts_mut(pa.0 as *mut u8, 4096) }
18     }
19     /// Get the mutable reference of physical address
20     pub fn get_mut<T>(&self) -> &'static mut T {
21         let pa: PhysAddr = (*self).into();
22         pa.get_mut()
23     }
24 }

```

建立和拆除虚实地址映射关系

这个逻辑较为简单，就是使用VPN的三级索引去查询PageTable，这里需要注意的几个点是：

1. RV39 中 VPN 是三个 9bit 的 `usize`；
2. VPN 不是 VA，他和 `offset` 结合才是 VA；
3. `ppn.get_pte_array()` 获取的是通过 `root_ppn` 读取的连续的 PTE 对象。

```

1  impl VirtPageNum {
2      /// Get the indexes of the page table entry
3      pub fn indexes(&self) -> [usize; 3] {
4          let mut vpn = self.0;
5          let mut idx = [0usize; 3];
6          for i in (0..3).rev() {
7              idx[i] = vpn & 0x1FF;
8              vpn >>= 9;
9          }
10         idx
11     }
12 }
13
14 impl PageTable {
15     /// Find PageTableEntry by VirtPageNum, create a frame for a 4KB page table if
16     not exist
17     fn find_pte_create(&mut self, vpn: VirtPageNum) -> Option<&mut PageTableEntry> {
18         let idxs = vpn.indexes();
19         let mut ppn = self.root_ppn;
20         let mut result: Option<&mut PageTableEntry> = None;
21         for (i, idx) in idxs.iter().enumerate() {
22             let pte = &mut ppn.get_pte_array()[*idx];
23             if i == 2 {
24                 result = Some(pte);
25             }
26         }
27     }
28 }

```

```

24         break;
25     }
26     if !pte.is_valid() {
27         let frame = frame_alloc().unwrap();
28         *pte = PageTableEntry::new(frame.ppn, PTEFlags::V);
29         self.frames.push(frame);
30     }
31     ppn = pte.ppn();
32 }
33 result
34 }
35
36 }

```

此外，我们在一些场景下可能需要访问非当前地址空间的页表（例如，在系统调用时内核态需要访问用户态的内存空间），此时我们需要一个方法，能查询到其他地址空间的页表但是不影响其所有权，我们提供了下面这个方法：

```

1  impl PageTable {
2      /// Temporarily used to get arguments from user space.
3      pub fn from_token(satp: usize) -> Self {
4          Self {
5              root_ppn: PhysPageNum::from(satp & ((lsize << 44) - 1)),
6              frames: Vec::new(),
7          }
8      }
9      /// get the page table entry from the virtual page number
10     pub fn translate(&self, vpn: VirtPageNum) -> Option<PageTableEntry> {
11         self.find_pte(vpn).map(|pte| *pte)
12     }
13 }

```

内核与应用的地址空间

`PageTable` 以页为单位维护虚拟内存到物理内存的映射关系，但是在实际的应用中，每个应用都会维护一个自己的页表（包括内核）。而这个应用间独立的页表由 `地址空间` 来维护。

实现地址空间抽象

逻辑段：一段连续地址的虚拟内存

```

1  /// map area structure, controls a contiguous piece of virtual memory
2  pub struct MapArea {
3      vpn_range: VPNRange,
4      data_frames: BTreeMap<VirtPageNum, FrameTracker>,
5      map_type: MapType,
6      map_perm: MapPermission,
7  }

```

1. `MapArea` 代表一段连续的 虚拟内存区间（逻辑段）；

- `vpn_range`: 保存该段包含的连续 `VirtPageNum` 区间，明确虚拟内存的范围（段内连续）；
 - `data_frames`: 仅当逻辑段采用 `MapType::Framed` 映射时有效，是存储“虚拟页（VPN）→ 物理页帧（FrameTracker）”的键值对容器（`BTreeMap`）。这些物理页帧用于存放实际数据（而非多级页表的中间节点），核心作用是持有物理页的所有权；
 - `map_type`: 指定该段虚拟内存的映射方式（如恒等映射、动态映射，后续可扩展文件映射等）；
 - `map_perm`: 指定该段的访问权限（R/W/X/ 用户态可访问，即 U 位），与页表项（PTE）的权限一致；
2. `VPNRange`: 通过 `SimpleRange` 组织连续的 `VirtPageNum` 区间，为 `vpn_range` 提供底层支持；
- 这些逻辑段的虚拟地址通常是连续的，因此用 `vpn_range` 明确其范围；
 - 不同段的访问权限不同（`.data` 为 R/W、`.rodata` 为 R、`.text` 为 R/X），因此用 `map_perm` 定义权限；
 - 不同段的映射方式不同（用户 `heap` 需动态映射、内核 `.text/.data` 需恒等映射），因此用 `map_type` 区分；
 - `MapArea` 管理一段连续的虚拟内存及其对应的物理内存，需对二者的生命周期负责：虚拟内存的范围由 `vpn_range` 界定，物理内存的所有权由 `data_frames` 持有（销毁 `MapArea` 时自动回收物理页）；同时 `data_frames` 提供“VPN→FrameTracker”的高效反向查询，比遍历页表更快；
 - `data_frames` 与 `PageTable` 的核心差别：`data_frames` 是物理内存的“所有者”（管归属、管生命周期），`PageTable` 是地址翻译的“查询工具”（管 VA→PA 映射、管权限检查），二者分工协作，缺一不可。

地址空间：一系列有关联的逻辑段

```
1  /// address space
2  pub struct MemorySet {
3      page_table: PageTable,
4      areas: Vec<MapArea>,
5  }
```

1. `MemorySet` 一个有关联但是不一定连续的逻辑段的集合。前面提到一个进程里会有 `.data`，`.text`，`.rodata` 等逻辑段，这些逻辑段用 `MapArea` 来表示，而 `MemorySet` 是这些逻辑段的集合。
- `page_table` 页表，每个进程都有自己的独立的页表。不同进程的同一个 VA 会通过页表映射到独立的 PA；
 - `areas` 是 `MapArea` 的集合，包含了一个程序的所有逻辑段；

```
1  impl MemorySet {
2      /// 1. Map a `MapArea` to the page table.
3      /// 2. If any input files are provided, copy the data from the input files into
4      the newly initialized `MapArea`.
5      /// 3. Push MapArea into `MemorySet`
6      fn push(&mut self, mut map_area: MapArea, data: Option<&[u8]>) {}
7
8      /// Create a `MapArea` spanning the virtual address (VA) range from `start_va` to
9      `end_va` with the given `permission`.
10     pub fn insert_framed_area(
```

```

9      &mut self,
10      start_va: VirtAddr,
11      end_va: VirtAddr,
12      permission: MapPermission,
13  ) {}
14
15  /// Without kernel stacks.
16  pub fn new_kernel() -> Self {}
17
18
19  /// Include sections in elf and trampoline and TrapContext and user stack,
20  /// also returns user_sp_base and entry point.
21  pub fn from_elf(elf_data: &[u8]) -> (Self, usize, usize) {}
22 }

```

MemorySet

- `push` :
 1. 为 `map_area` 中所有的 `vpn` 分配对应的 `ppn`，并更新页表，注意，此时我们只是初始化了 **VPN -> PPN** 的映射关系，此时数据是没有写入到内存的。
 2. 如果提供了 `data` 作为输入，我们将 `data` 复制到主存。注意，在这个时刻我们已经开始需要使用 **PageTable**来映射**VPN -> PPN**了。
 3. 将构造好的 `map_area` 加入我们的地址空间 -- 也就是 `MemorySet`。
- `insert_framed_area` 创建一个从 `start_va` 开始到 `end_va` 结束的 `MapArea`。

内核地址空间

启用分页模式下，内核代码的访存地址也会被视为一个虚拟地址并需要经过 MMU 的地址转换，因此我们也需要为内核对应构造一个地址空间，它除了仍然需要允许内核的各数据段能够被正常访问之后，还需要包含所有应用的内核栈以及一个跳板 (Trampoline)。

在开始梳理代码之前，我们需要先了解以下内容：

- [sv39模式下的地址空间](#)
- [应用栈和应用内核栈以及内核栈](#)

`new_kernel()` 方法将 `.text/.rodata/.data/.bss` 以及 `内核->内存结束` 的这五个逻辑段初始化完成。

这里最需要注意的是：我们所有的 `memory_set.push()` 方法都是不带 `data` 参数的，这是因为我们这里使用的恒等映射，他所访问的内存区域本身就是存在数据的。

```

1  // os/src/mm/memory_set.rs
2
3  extern "C" {
4      fn stext();
5      fn etext();
6      // ...
7  }
8
9  impl MemorySet {

```

```

10     /// Without kernel stacks.
11     pub fn new_kernel() -> Self {
12         let mut memory_set = Self::new_bare();
13         // map trampoline
14         memory_set.map_trampoline();
15         // map kernel sections
16         println!("[main][kernel] Hello, world!");
17         // ...
18         println!("mapping .text section");
19         memory_set.push(MapArea::new(
20             (stext as usize).into(),
21             (etext as usize).into(),
22             MapType::Identical,
23             MapPermission::R | MapPermission::X,
24             None);
25         // ...
26         memory_set
27     }
28 }

```

上面的代码我们可以把LOG日志打开测试：

```
1 make run BASE=4 TEST=4 LOG=TRACE
```

可以看到我们的输出，起始位置和我们 linker.ld 的声明是一致的：

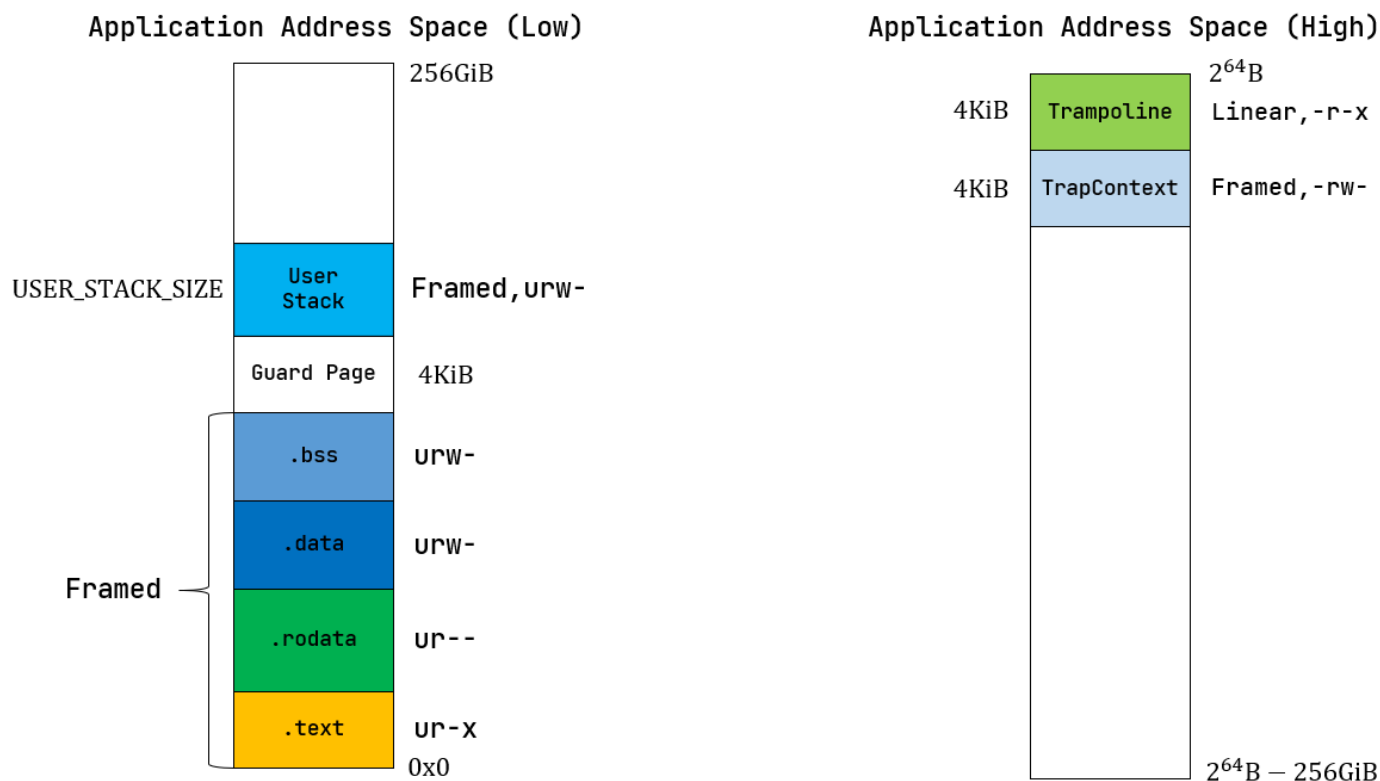
```

1 [TRACE] [main][kernel] .text [0x80200000, 0x8020c000)
2 [DEBUG] [main][kernel] .rodata [0x8020c000, 0x80210000)
3 [ INFO] [main][kernel] .data [0x80210000, 0x8023a000)
4 [ WARN] [main][kernel] boot_stack top=bottom=0x8024a000, lower_bound=0x8023a000
5 [ERROR] [main][kernel] .bss [0x8024a000, 0x8224b000)

```

应用地址空间

在开始之前，我们需要了解一下 [trampoline](#)



应用地址空间的初始化逻辑如下：

1. 初始化高256GiB：

1. 初始化 `trampoline`，trampoline 位于高256GiB的最高点起始位置，仅仅使用一页；
2. 初始化 `TrapContext`，他在trampoline的下方，也仅仅使用一页；

2. 初始化低256GiB：

1. 初始化 `.text/.rodata/.data/.bss`：解析 `elf` 文件，将elf文件中的所有 program headers 复制到内核中：
 1. 解析elf，为每个ph创建一个MapArea；
 2. 将ph复制到我们创建的MapArea中，并初始化对应的权限；
 3. 这里一定需要注意的是，我们在 `xmas_elf::ElfFile::new(elf_data)` 的时候已经将数据加载到物理内存了，为什么还需要复制到我们对应的MapArea呢？
 - 现在读取到的数据虽然已经在物理内存中了，但是他只是在这个函数下的一段临时的数据，我们需要把他转换为一个生命周期和应用一致的程序，也就是MapArea；如果我们不把他转换为MapArea，那么rust编译器会保证在该函数退出后他对应的物理内存被回收；
 - 读取到数据之后，`[ph.virtual_addr(), ph.virtual_addr() + ph.mem_size())` 这段虚拟内存保证是可用的。我们其实也可以选择自己重新为它生成一个新的虚拟内存片地址，但是意义不大。
2. 初始化 `user stack`；
3. 初始化 `sbrk`；

初始化 trampoline

下面的代码中，在高256G中初始化了一页作为trampoline，我们需要注意以下几个点：

1. `strampoline` 在 `linker.ld` 中声明，他紧跟在 `*(.text.entry)` 进行4K对齐之后；
2. `strampoline` 自身的内容也需要4K对齐，这样可以正好去适配我们的一个内存页；
3. `strampoline` 的大小不能超过4K，否则在map的时候将出现数据缺失；
4. `trampoline` 的范围是高256的最高位开始向下计算一页。

```
1      .text : {
2          *(.text.entry)
3          . = ALIGN(4K);
4          strampoline = .;
5          *(.text.trampoline);
6          . = ALIGN(4K);
7          *(.text .text.*)
8      }
```

```
1  /// config.rs
2  /// The virtual address of the trampoline.
3  /// The trampoline is placed at the top of the high 256 GiB virtual address space,
4  /// with its starting address set to `usize::MAX - PAGE_SIZE + 1`.
5  pub const TRAMPOLINE: usize = usize::MAX - PAGE_SIZE + 1;
6
7  /// memory_set.rs
8  extern "C" {
9      fn strampoline();
10 }
11
12 /// memory_set.rs
13 impl MemorySet {
14     /// Mention that trampoline is not collected by areas.
15     fn map_trampoline(&mut self) {
16         self.page_table.map(
17             VirtAddr::from(TRAMPOLINE).into(),
18             PhysAddr::from(strampoline as usize).into(),
19             PTEFlags::R | PTEFlags::X,
20         );
21     }
22 }
```

从elf初始化.text/.rodata/.data/.bss

APPENDIX: [elf文件的VA和PA](#)

1. 遍历elf文件内的 `program headers`。这样，ph会被加载到内存中，我们就可以用他们来创建出一个 `MapArea`了；
2. 在创建 `MapArea` 之后，有非常重要的一个步骤，那就是把我们elf的逻辑段数据映射到我们新创建的 **MapArea**。不同于 `trampoline`, `TrapContext`, `user_stack`, `sbrk` 这几个逻辑段，elf的逻辑数据段是有初始值的 -- 也就是elf文件里的值。我们现在只是为elf文件创建了映射，映射指向的物理内存数据仍然不存在。
3. 在 `push()` 的时候将数据作为参数传入，初始化elf的逻辑段到虚拟内存 `(&elf.input[ph.offset() as`


```
usize..(ph.offset() + ph.file_size()) as usize]]。
```

```
1 fn map_elf(memory_set: &mut MemorySet, elf: & xmas_elf::ElfFile) -> VirtPageNum {
2     let elf_header = elf.header;
3     let magic = elf_header.pt1.magic;
4     assert_eq!(magic, [0x7f, 0x45, 0x4c, 0x46], "invalid elf!");
5     let ph_count = elf_header.pt2.ph_count();
6     let mut max_end_vpn = VirtPageNum(0);
7     for i in 0..ph_count {
8         let ph = elf.program_header(i).unwrap();
9         if ph.get_type().unwrap() == xmas_elf::program::Type::Load {
10             // Notice that the elf file has been loaded into virtual memory
11             let start_va: VirtAddr = (ph.virtual_addr() as usize).into();
12             let end_va: VirtAddr = ((ph.virtual_addr() + ph.mem_size()) as
13             usize).into();
14             let mut map_perm = MapPermission::U;
15             let ph_flags = ph.flags();
16             if ph_flags.is_read() {
17                 map_perm |= MapPermission::R;
18             }
19             if ph_flags.is_write() {
20                 map_perm |= MapPermission::W;
21             }
22             if ph_flags.is_execute() {
23                 map_perm |= MapPermission::X;
24             }
25             let map_area = MapArea::new(start_va, end_va, MapType::Framed, map_perm);
26             max_end_vpn = map_area.vpn_range.get_end();
27             // init MapArea and copy program header into the newly created MapArea
28             memory_set.push(
29                 map_area,
30                 Some(&elf.input[ph.offset() as usize..(ph.offset() + ph.file_size())
31                 as usize]),
32             );
33         }
34     }
35     max_end_vpn
36 }
```

其他部分初始化

除了 trampoline 之外，其他的逻辑基本不复杂，这里就不做赘述。

基于地址空间的分时多任务

建立并开启基于分页模式的虚拟地址空间

机器启动时是 **M** 模式，SBI加载并初始化完成后进入 **S** 模式，随后内核初始化页表等信息后，修改 **satp** 开启分页模式。

创建内核地址空间

`lazy_static!` 的描述参考 [lazy_static!](#)

```
1 // os/src/mm/memory_set.rs
2
3 lazy_static! {
4     pub static ref KERNEL_SPACE: Arc<UPSafeCell<MemorySet>> = Arc::new(unsafe {
5         UPSafeCell::new(MemorySet::new_kernel()
6     )});
7 }
```

创建内核地址空间需要有三部：

1. 为rust初始化一个 HEAP；
2. 为kernel初始化一个物理内存分配器；
3. 修改 `satp` 开启分页模式。

```
1 // os/src/mm/mod.rs
2
3 pub use memory_set::KERNEL_SPACE;
4
5 pub fn init() {
6     heap_allocator::init_heap();
7     frame_allocator::init_frame_allocator();
8     KERNEL_SPACE.exclusive_access().activate();
9 }
```

我们先通过 `init_heap()` 初始化了堆，这里初始化的过程可以参考 [堆的初始化](#)

随后我们初始化 `FRAME_ALLOCATOR`，这个模块绑定了 `[ekernel, MEMORY_END)` 这段 物理内存空间；

```
1 /// `frame_allocator.rs`
2 /// initiate the frame allocator using `ekernel` and `MEMORY_END`
3 pub fn init_frame_allocator() {
4     extern "C" {
5         fn ekernel();
6     }
7     FRAME_ALLOCATOR.exclusive_access().init(
8         PhysAddr::from(ekernel as usize).ceil(),
9         PhysAddr::from(MEMORY_END).floor(),
10    );
11 }
```

```
1 OUTPUT_ARCH(riscv)
2 ENTRY(_start)
3 BASE_ADDRESS = 0x80200000;
4
5 SECTIONS
```

```

6 {
7     // ...
8     . = BASE_ADDRESS;
9     . = ALIGN(4K);
10    ebss = .;
11    ekernel = .;
12
13    /DISCARD/ : {
14        *(.eh_frame)
15    }
16 }

```

最后，我们开启了分页模式：

1. `self.page_table.token()` 按照CSR要求开启 `sv39` 分页模式；
2. `sfence.vma` 将快表清空。



Figure 4.14: RV64 Supervisor address translation and protection register `satp`, for MODE values Bare, Sv39, and Sv48.

```

1  impl MemorySet {
2      /// Change page table by writing satp CSR Register.
3      pub fn activate(&self) {
4          let satp = self.page_table.token();
5          unsafe {
6              satp::write(satp);
7              asm!("sfence.vma");
8          }
9          info!("[kernel]: enable kernel space.")
10     }
11 }

```

trampoline的实现

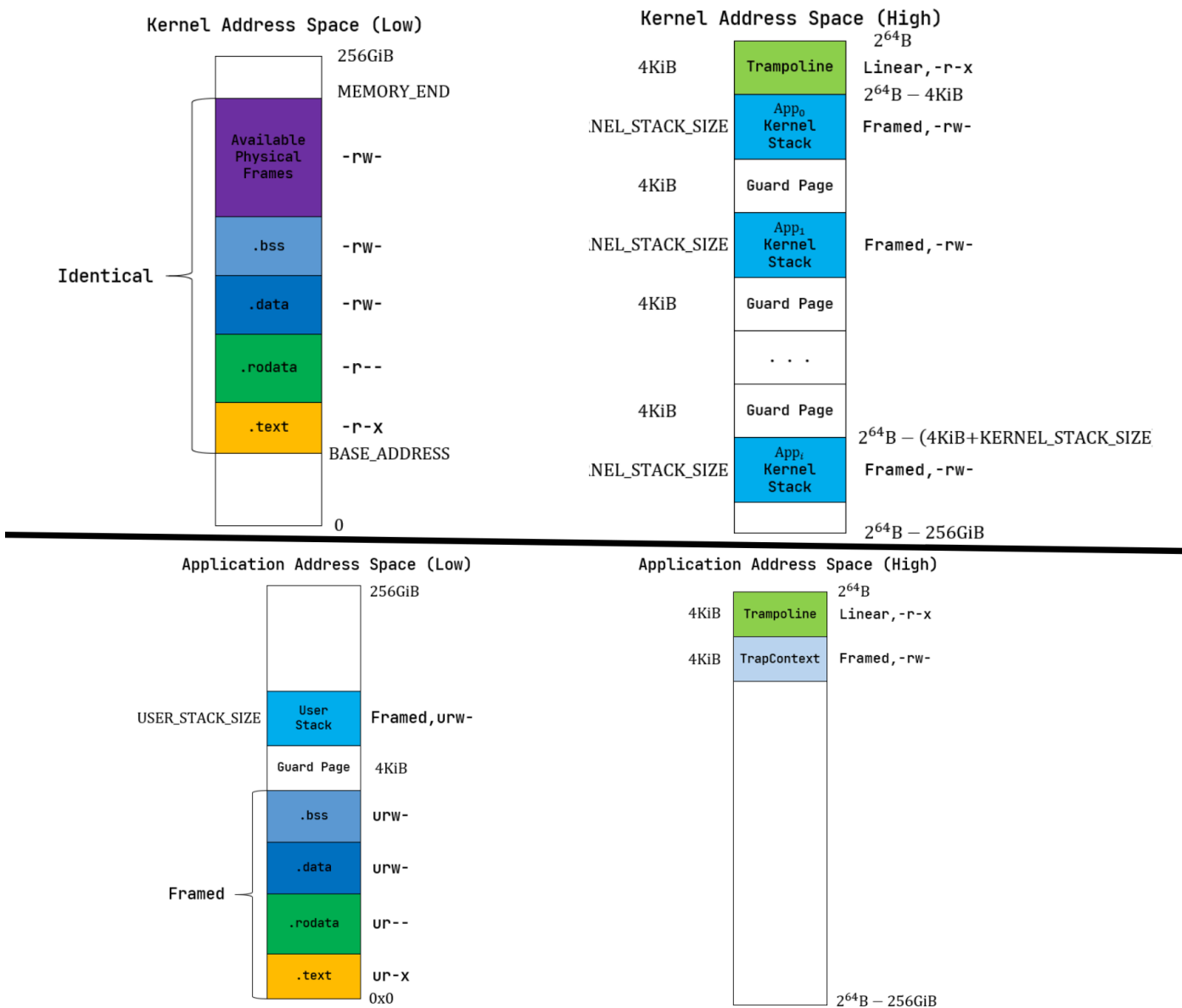
在 [trampoline](#) 中我们分析了为什么需要这样一个中间层，下面我们来实现他的具体逻辑。

在开始之前，我们可以讨论一下实现内核的两个方式：

1. 每个进程的地址空间都会将逻辑段分为 `内核段` 和 `用户段` 两个部分；
2. 每个进程的地址空间值包含 `用户段`，他们共享一个 `内核段`。

第一种方式的好处是，用户和内核的切换不需要切换地址空间，只需要在进程之间切换内核段，这样实现较为简单。但是带来的问题是，每个进程都需要存储一份冗余的内核代码，浪费了大量的内存。

第二种方式节省内存，但是实现较为复杂。



可以看到，我们的TrapContext位于 应用地址空间的次高页面 而不是内核地址空间。原因在于，假如我们将其放在内核栈中，在保存Trap上下文之前我们必须先切换到内核地址空间，这就需要我们做两件事：

1. 将内核地址空间的 token 写入 satp 寄存器，注意 satp 不是单纯的页表地址，因为它还指定了CPU的分页模式；
2. 之后我们还需要有一个通用寄存器保存内核栈栈顶的位置，这样才能以它为基址保存 Trap 上下文。

在保存Trap上下文之前我们必须完成这两项工作。然而，我们无法在不破坏任何一个通用寄存器的情况下做到这一点。

因为事实上我们需要用到内核的两条信息：内核地址空间的 token 还有应用内核栈顶的位置，硬件却只提供一个 `sscratch` 可以用来进行周转。所以，我们不得不将Trap上下文保存在应用地址空间的一个虚拟页面中以避免切换到内核地址空间才能保存。

```

1  #[repr(C)]
2  #[derive(Debug)]
3  pub struct TrapContext {
4      pub x: [usize; 32],
5      pub sstatus: Sstatus,
6      pub sepc: usize,
7      pub kernel_satp: usize,
8      pub kernel_sp: usize,
9      pub trap_handler: usize,
10 }

```

老的 TrapContext 如下定义：

```

1  #[repr(C)]
2  #[derive(Debug)]
3  pub struct TrapContext {
4      pub x: [usize; 32],
5      pub sstatus: Sstatus,
6      pub sepc: usize,
7  }

```

相对于之前，额外增加了 `kernel_satp`，`kernel_sp`，`trap_handler` 三个字段：

- `kernel_satp` 表示内核地址空间的 token；
- `kernel_sp` 表示当前应用在内核地址空间中的内核栈栈顶的虚拟地址；
- `trap_handler` 表示内核中 trap handler 入口点的虚拟地址。

QA

TrapContext的初始化

再回忆一次 `TaskContext` 和 `TrapContext` 的区别：

`TrapContext` 是应用和内核切换时的上下文，而 `TaskContext` 是应用之间切换的上下文切换。

`TrapContext` 是不可预测的，所以他需要保存全部的寄存器。而 `TaskContext` 是用户基于时钟或者主动调用的，是可预测的，所以只需要保存 callee-saved 寄存器。

`TrapContext` 是和应用强绑定的，内核在初始化应用时，会为每一个应用初始化一个 `TaskControlBlock(TCB)`，而 `TrapContext` 就是 TCB 的一部分。

```

1  /// The task control block (TCB) of a task.
2  pub struct TaskControlBlock {
3      /// ...
4      /// The phys page number of trap context
5      pub trap_cx_ppn: PhysPageNum,
6  }

```

可以看到，我们的 `TrapContext` 实际上是单独的一个内存页，`TrapContext` 的虚拟地址是 `0xffff_ffff_ffff_e000` (`TRAP_CONTEXT_BASE`)，trampoline 的虚拟地址是 `0xffff_ffff_ffff_f000` (`TRAMPOLINE`) —— 两者是连续的两个高地址页，`TrapContext` 紧邻 trampoline 的“下方”（低地址侧）-- 对于任意一个进程，它的 `TrapContext` 的虚拟地址是固定的！

```
1  impl TaskControlBlock {
2      pub fn new(elf_data: &[u8], app_id: usize) -> Self {
3          // memory_set with elf program headers/trampoline/trap context/user stack
4          let (memory_set, user_sp, entry_point) = MemorySet::from_elf(elf_data);
5          let trap_cx_ppn = memory_set
6              .translate(VirtAddr::from(TRAP_CONTEXT_BASE).into())
7              .unwrap()
8              .ppn();
9      }
10 }
```

而它被当做一个逻辑段，在用户内存地址初始化的时候被初始化：

```
1  impl MemorySet {
2      pub fn from_elf(elf_data: &[u8]) -> (Self, usize, usize) {
3          // map TrapContext
4          memory_set.push(
5              MapArea::new(
6                  TRAP_CONTEXT_BASE.into(),
7                  TRAMPOLINE.into(),
8                  MapType::Framed,
9                  MapPermission::R | MapPermission::W,
10             ),
11             None,
12         );
13     }
14 }
```

sscratch

先说结论，`sscratch` 是 `trap_return` 函数第一次调用的时候，使用 `TRAP_CONTEXT_BASE` 初始化的一个指向 `TrapContext` 的指针。

在用户进程启动时，内核会为每个进程初始化 `TaskControlBlock`：

```

1 lazy_static! {
2     /// a `TaskManager` global instance through lazy_static!
3     pub static ref TASK_MANAGER: TaskManager = {
4         // ...
5         for i in 0..num_app {
6             tasks.push(TaskControlBlock::new(get_app_data(i), i));           /// init
7         }
8         // ...
9     };
10 }

```

而在初始化 TaskControlBlock 的过程中，会初始化 TaskContext

```

1 impl TaskControlBlock {
2     pub fn new(elf_data: &[u8], app_id: usize) -> Self {
3         let task_control_block = Self {
4             task_status,
5             task_cx: TaskContext::goto_trap_return(kernel_stack_top),           ///
6             init task context
7                 memory_set,
8                 trap_cx_ppn,
9                 base_size: user_sp,
10                heap_bottom: user_sp,
11                program_brk: user_sp,
12            };
13 }

```

这个时候，会为 TaskContext 中的 ra 赋值：

```

1 impl TaskContext {
2     pub fn goto_trap_return(kstack_ptr: usize) -> Self {
3         Self {
4             ra: trap_return as usize,
5             /// init return address
6             sp: kstack_ptr,
7             s: [0; 12],
8         }
9     }
}

```

这里，ra 的值就是 trap_return：也就是说，当内核第一次调度该进程的时候，内核在执行完内核态的所有指令后，会跳转到 trap_return。

```

1 #[no_mangle]
2 pub fn trap_return() -> ! {
3     // set stvec to trampoline
4     set_user_trap_entry();

```

```

5      // get trap context pointer
6      let trap_cx_ptr = TRAP_CONTEXT_BASE;
7      // get the user page table addr with other satp fields
8      let user_satp = current_user_token();
9      extern "C" {
10         fn __alltraps();
11         fn __restore();
12     }
13     //
14     let restore_va = __restore as usize - __alltraps as usize + TRAMPOLINE;
15     // trace!("[kernel] trap_return: ..before return");
16     unsafe {
17         asm!(
18             "fence.i",
19             "jr {restore_va}",          // jump to new addr of __restore asm function
20             restore_va = in(reg) restore_va,
21             in("a0") trap_cx_ptr,      // a0 = virt addr of Trap Context
22             in("a1") user_satp,        // a1 = phy addr of usr page table
23             options(noreturn)
24         );
25     }
26 }

```

`trap_return` 中初始化了 `stvec`，并使用 `trap_cx_ptr -> TRAP_CONTEXT_BASE` 和 `user_satp` 作为参数调用了方法 `__restore`，值得注意的是：

1. 我们使用的是 `jr restore_va` 指令而不是 `call __restore`；
2. `restore_va` 也不是直接使用 `jr __restore`；

这个逻辑可以在 [restore_va](#) 中找到具体解析，这里我们先看 `sscratch` 相关的内容。

在 `__restore` 中，我们将 `a0 = trap_cx_ptr` 赋值给了 `sscratch`，于是初始化完成。

restore_va

在开始之前，我们需要先了解 `linker.ld` 中声明的地址和物理地址的关系 -- 是的，`linker.ld` 中的地址是虚拟地址。细节参考 [linker.ld](#)

```

1  pub fn trap_return() -> ! {
2      extern "C" {
3          fn __alltraps();
4          fn __restore();
5      }
6      //
7      let restore_va = __restore as usize - __alltraps as usize + TRAMPOLINE;
8      // trace!("[kernel] trap_return: ..before return");
9      unsafe {
10         asm!(
11             "fence.i",
12             "jr {restore_va}",          // jump to new addr of __restore asm function
13             restore_va = in(reg) restore_va,

```



```

14         in("a0") trap_cx_ptr,      // a0 = virt addr of Trap Context
15         in("a1") user_satp,       // a1 = phy addr of usr page table
16         options(noreturn)
17     );
18 }
19 }

```

在内核态将控制权交还给用户态的时候，我们需要调用 `__restore`，而这里是通过 `restore_va` 这个奇怪的地址来跳转的，这是为什么？我们可以先看看 `restore_va` 这个值是怎么计算出来的。

我们可以找到，我们的 `strampoline` 实在 `linkder.ld` 上定义的，它被包含在了内核的 `.text` 逻辑段中：在加载内核代码的过程中，`strampoline` 这个 VA 被以恒等映射的方式映射到物理内存。

```

1  OUTPUT_ARCH(riscv)
2  ENTRY(_start)
3  BASE_ADDRESS = 0x80200000;
4
5  SECTIONS
6  {
7      . = BASE_ADDRESS;
8      skernel = .;
9
10     stext = .;
11     .text : {
12         *(.text.entry)
13         . = ALIGN(4K);
14         strampoline = .;          /// strampoline is defined
15     here
16         *(.text.trampoline);      /// .text.trampoline is
17     included here
18         . = ALIGN(4K);
19         *(.text .text.*)
20     }
21 }

```

在 `memory_set.rs` 中，我们提供了方法来映射 VA 到 PA：

```

1  extern "C" {
2      fn strampoline();
3  }
4
5  impl MemorySet {
6      /// Mention that trampoline is not collected by areas.
7      fn map_trampoline(&mut self) {
8          self.page_table.map(
9              VirtAddr::from(TRAMPOLINE).into(),
10             PhysAddr::from(strampoline as usize).into(),
11             PTEFlags::R | PTEFlags::X,
12         );
13     }
14 }

```

随后，我们在加载内核以及内核加载应用程序的时候，都调用 `map_trampoline` 这个函数，将 `TRAMPOLINE` 这个 VA 映射到了同一个 PA。

再回到 `__alltraps` 和 `__restore`，在 `trap.S` 中，我们声明 `.text.trampoline` 逻辑段和这两个方法：

```
1  .section .text.trampoline
2  .globl __alltraps
3  .globl __restore
4  .align 2
5  __alltraps:
6      # ...
7  __restore:
8      # ...
```

这里存在的一个问题是：`__alltraps` 和 `__restore` 是被引入到了 `linker.ld` 的 `.text` 段，并且它加载时尚未开启分页模式，也就是 `__alltraps` 和 `__restore` 标签的值是一个 PA。此时，当我们执行 `call __alltraps` 或者 `call __restore` 时将发生异常，因为此时我们已经开启了分页模式。因此，我们只能手动的计算这个他们的值。

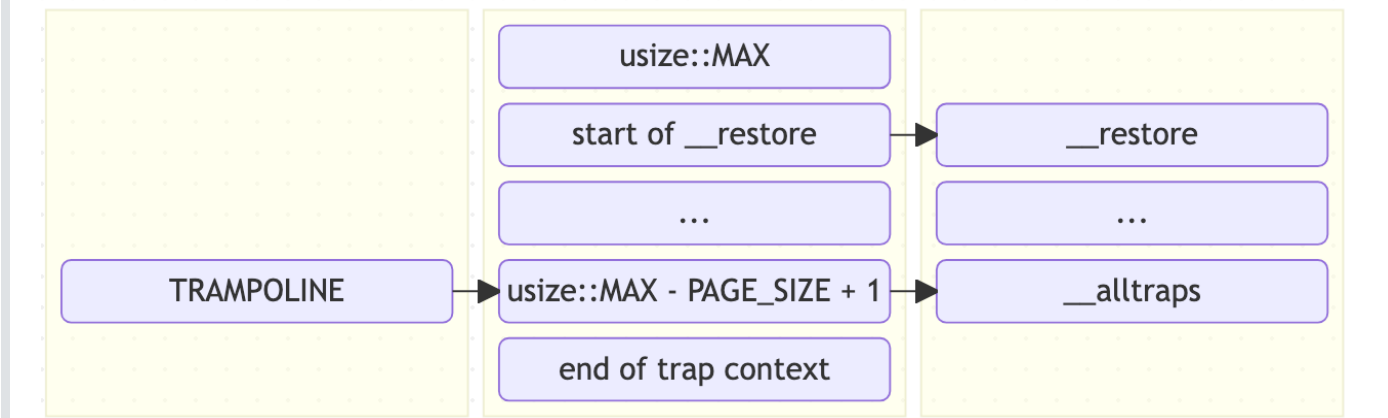
那么，我们现在可以知道几个信息：VA `TRAMPOLINE` 指向了 PA `strampoline`：

- 访问 `__alltraps` 我们可以直接使用 VA `TRAMPOLINE`；
- 访问 `__restore` 我们可以使用 $VA\ TRAMPOLINE + len(__alltraps) = TRAMPOLINE + (__restore - __alltraps)$ 。

在上面的地址计算中，有一个需要注意的地方是，画内存布局图时从高到低画，但代码实际从低地址往高地址加载（trampoline 代码从下往上）

以 `trampoline` 为例，trampoline 的指针是 `TRAMPOLINE`，那么实际的加载的数据是从 `TRAMPOLINE -> TRAMPOLINE + PAGE_SIZE - 1` 这个方向去加载的。

也就是说，如果把 trampoline 的代码也画到内存布局图上的话，它的代码是从下往上的。



linker.ld

我们的 `linker.ld` 一般是如下形式，而其中声明的地址例如 `BASE_ADDRESS = 0x80200000` 都是虚拟地址 -- 不管是内核态的 `linker.ld` 还是用户态的 `linker.ld`。这也是为什么我们在之前处理 `elf` 文件的时候，可以通过 `ph.virtual_addr()` 获取到一个逻辑段的虚拟地址的原因（最简单的理解是，物理地址是运行时才有的概念，在静态文件中根本就不可能知道任何物理地址的信息）。

```
1 OUTPUT_ARCH(riscv)
2 ENTRY(_start)
3 BASE_ADDRESS = 0x80200000;
4
5 SECTIONS
6 {
7     . = BASE_ADDRESS;
8     skernel = .;
9 }
```

在 `.cargo/config.yml` 中，我们指定了该 `linker.ld`：

```
1 [build]
2 target = "riscv64gc-unknown-none-elf"
3
4 [target.riscv64gc-unknown-none-elf]
5 rustflags = [
6     "-Clink-arg=-Tsrc/linker.ld", "-Cforce-frame-pointers=yes"
7 ]
```

然而，在编译的过程中，我们在加载这个程序到内存时是一个裸机程序，他和其他的应用程序不一样。他是从 **M** 模式启动加载的，此时 PC 尚未开启分页模式 -- 也就是我们的 `VA` 会直接被映射到对应的 `PA`。

当内核加载完成，开启分页模式后，此时再加载应用程序，此时的 `VA` 已经在加载阶段被我们映射到了 `PA`。

M 模式启动加载

1. RISC-V 复位后，CPU 进入 **M 模式**，执行 bootloader（通常是开源的 OpenSBI，或简易 bootloader）；
2. bootloader 完成硬件初始化（如内存初始化、串口配置）后，会将内核 ELF 加载到物理地址 `0x80200000`；
3. 随后 bootloader 切换到 **S 模式**（监督模式），并跳转到内核的入口地址（`_start`，虚拟地址 = 物理地址 = `0x80200000`）；
4. 内核后续在 S 模式运行（管理页表、处理 Trap、调度应用），M 模式仅用于 bootloader 初始化，内核运行时不涉及 M 模式。

再回头看我们的 `entry.asm` 文件，这里声明了 `.text.entry` 段和 `_start`：

```
1     .section .text.entry                # 这里声明了 .text.entry
2     .globl _start
3 _start:
4     la sp, boot_stack_top
5     call rust_main                     # 这就是内核的入口地址
```

而 `linker.ld` 中将 `text.entry` 加载到了 `.text` 的起始位置：

```
1 OUTPUT_ARCH(riscv)
2 ENTRY(_start)
3 BASE_ADDRESS = 0x80200000;
4
5 SECTIONS
6 {
7     . = BASE_ADDRESS;
8     skernel = .;
9
10    stext = .;
11    .text : {
12        *(.text.entry)                                # 这里将 .text.entry 加载到
13    .text
14        . = ALIGN(4K);
15        strampoline = .;
16        *(.text.trampoline);
17        . = ALIGN(4K);
18        *(.text .text.*)
19    }
```

堆的初始化

在本项目中，`HEAP_ALLOCATOR` 靠 Rust 的 `#[global_allocator]` 编译器属性“注册为全局堆分配器”，`HEAP_SPACE` 是堆的“物理内存载体”，通过 `init_heap()` 手动绑定到分配器——两者无需外部显式引用，靠编译器约定和初始化逻辑协同工作，最终让内核的 `Box`/`Vec`/`Arc` 等动态分配操作自动路由到这个堆。

三个核心组件

组件	角色	关键特性
HEAP_ALLOCATOR	全局堆分配器实例 (<code>buddy_system_allocator</code> 的伙伴系统分配器)	被 <code>#[global_allocator]</code> 标记，编译器会自动将所有堆分配请求路由到它； <code>LockedHeap</code> 带锁，支持多线程安全。
HEAP_SPACE	内核堆的“内存空间载体”（一块静态数组）	本质是 <code>.bss</code> 段的连续内存（编译期分配大小 <code>KERNEL_HEAP_SIZE</code> ，加载时操作系统清零），是堆分配的“物理基础”。
<code>init_heap()</code>	绑定“载体”和“分配器”的初始化函数	手动将 <code>HEAP_SPACE</code> 的地址和大小告诉 <code>HEAP_ALLOCATOR</code> ，让分配器知道“管理哪块内存”。

`#[global_allocator]`

Rust 没有内置的内核堆，必须手动实现 / 指定“全局分配器”——`#[global_allocator]` 就是编译器提供的“注册约定”：

只要有一个静态变量被 `#[global_allocator]` 标记，且该变量实现了 `core::alloc::GlobalAlloc` trait（`LockedHeap` 已实现），编译器就会将所有“动态内存分配操作”（如 `Box::new()`、`Vec::push()`、`Arc::new()`、`String::from()` 等）路由到这个分配器的 `alloc/dealloc` 方法。

HEAP_SPACE和HEAP_ALLOCATOR

HEAP_ALLOCATOR 初始化时是 `LockedHeap::empty()`（空分配器，不知道要管理哪块内存），而 HEAP_SPACE 是一块“静态分配的连续内存”（.bss 段），`init_heap()` 的作用就是把两者“绑在一起”：

lazy_static

`lazy_static!` 是 Rust 社区提供的“延迟初始化静态变量”宏（属于 `lazy_static` crate），核心解决“无法在编译期初始化复杂静态变量”的问题

`MemorySet::new_kernel()` 是运行时执行的函数（需要初始化页表、映射内核段等），不能直接用 `static` 定义（`static` 要求编译期常量初始化）。

`lazy_static!` 的实现原理是：

- 编译期：宏会生成一个隐藏的静态变量（标记为 `lazy`），并分配一小块空间在 `.data` 段（或 `.bss` 段，取决于是否有初始值），用于存储“初始化状态”（是否已创建实例）和“实例指针”；
- 运行时：首次访问 `KERNEL_SPACE` 时，会检查初始化状态——若未初始化，执行 `MemorySet::new_kernel()` 创建实例，将实例的所有权交给 `Arc`，再把 `Arc` 指针存入之前分配的 `.data` 段空间；后续访问直接复用该实例，不再重复初始化。

`lazy_static!` 会包含三个模块：

1. 实例指针：用于存储后续初始化完成后，`Arc<UPSafeCell>` 的指针（初始值为 `null` 或占位符）；
2. 初始化状态标记：一个原子变量（如 `AtomicBool` 或枚举 `Uninit/Init`），标记实例是否已初始化（初始状态为“未初始化”）；
3. 同步锁：用于多线程环境下的初始化互斥（避免多个线程同时触发初始化，导致实例重复创建）——这是 `lazy_static!` 默认提供的线程安全保障。

实际的执行过程可以概括为：

1. 获取同步锁，成功则进入 `<2>`
2. 查询初始化状态
 - 如果未初始化，则初始化对象，并给实例指针赋值；修改初始化状态标记；
 - 如果已经初始化，则直接使用对象；
3. 释放锁。

elf文件的VA和PA

在我们通过elf初始化 `.text/.rodata/.data/.bss` 逻辑段时，elf文件自身存在VA和PA，而我们需要把这个数据转换为MapArea，这个过程是怎么样的呢？

在分析这段代码的内存情况之前，我们需要知道目前内核的初始化状态：

1. `KERNEL_SPACE` 已经初始化，这意味着 `new_kernel()` 已经被调用，内核的页表已经初始化完成；
2. `MemorySet#activate()` 已经调用，这意味着分页访问已经开启；
3. 我们的目的是初始化用户地址空间，此时程序从头至尾均在内核态下执行。

那么整体的转换过程可以如下表示：

1. 内核读取 ELF 数据到内核地址空间：

1. 内核通过块设备驱动（如磁盘），将 ELF 文件读取到当前函数的栈 / 堆 / 全局缓冲区（`elf_data`）；
2. `elf_data` 的地址是内核 VA，通过内核页表翻译后，得到对应的物理地址 PA_ELF —— ELF 原始数据（.text/.data 等）存储在 PA_ELF 对应的物理内存中。

2. 解析 ELF 得到元数据，提取关键信息并复制；

3. 此时：我们的elf文件存储在 内核栈 上：

- 这些 ELF 原始数据（.text/.data 等）在「内核地址空间」的 VA 范围是 `[elf_data, elf_data + elf_data_size)`；
- 而 `[ph.virtual_addr(), ph.virtual_addr() + ph.mem_size())` 是这些数据在「用户地址空间」的「目标 VA 范围」—— 这个目标 VA 是链接阶段确定的，内核加载时会把数据拷贝到用户物理内存，并建立「用户目标 VA → 用户物理 PA」的映射。这也是为什么每一个应用程序能通过同一个 VA 访问到不同 PA 的根本原因 -- 这些数据将被映射到不同的 PA；

4. 最后，我们通过 `memory_set.push(map_area, elf.data)` 的方法，将这一份数据复制到了用户态的 `MemorySet` 中。注意，**MemorySet**是存在于内核态，对用户态完全透明的。

sbrk

`sbrk(n)` 是经典的 Unix 系统调用，功能是“扩展 / 收缩进程的堆空间”（或栈空间，取决于内核设计）：在我们的 `rCore` 中，`sbrk` 是紧跟着 `user_stack_top`。

```
1 fn map() {
2     user_stack_bottom += PAGE_SIZE;
3     let user_stack_top = user_stack_bottom + USER_STACK_SIZE;
4     memory_set.push(
5         MapArea::new(
6             user_stack_bottom.into(),
7             user_stack_top.into(),
8             MapType::Framed,
9             MapPermission::R | MapPermission::W | MapPermission::U,
10        ),
11        None,
12    );
13    // used in sbrk
14    memory_set.push(
15        MapArea::new(
16            user_stack_top.into(),
17            user_stack_top.into(),
18            MapType::Framed,
19            MapPermission::R | MapPermission::W | MapPermission::U,
20        ),
21        None,
22    );
23 }
```

我们可以通过调整 `sbrk` 来伸缩我们的栈大小：

- 当 $n > 0$ 时，向内核申请 n 字节的内存，堆 / 栈空间扩大；
- 当 $n < 0$ 时，释放 $|n|$ 字节的内存，堆 / 栈空间缩小；
- 返回值是扩展前的堆 / 栈顶指针（或出错时返回 -1）。

如果需要初始化 `sbrk`，可以在初始化的时候，设定一个 `start == end` 的 `MapArea`。

- 正常的 `MapArea` 是 $start_va < end_va$ （有实际地址范围），但这里 $start_va == end_va$ ，表示这是一个“空映射”——不占用实际虚拟地址空间，仅作为“预留标识”；
- 目的：告诉内核“`user_stack_top` 以下的区域（用户栈的预期扩展方向）是合法的栈空间”，后续 `sbrk` 扩展时，只需修改 `end_va` 扩大地址范围，无需重新创建 `MapArea`（避免重复映射冲突）；
- 补充：用户栈的扩展方向通常是“从高地址向低地址扩展”（`user_stack_top` 是栈顶高地址，扩展时栈底向低地址移动，`end_va` 减小），所以这里以 `user_stack_top` 为起点占位。

trampoline

trampoline 是“跳板”或“过渡代码段”——本质是一段固定在内核高地址空间（高 256GiB）的汇编代码，核心作用是解决“用户态（U 特权级）↔ 内核态（S 特权级）切换时的特权级 + 地址空间切换难题”

零、系统调用的流程

如果没有 `trampoline`，一次完整的系统调用流程如下所示：

1. 最开始，CPU 处于 `u` 模式，`satp` 指向用户态页表；
2. 调用 `ecall` 提升 CPU 到 `s` 模式，此时 `satp` 指向用户态页表；
3. **CPU 尝试通过 `stvec` 跳转到异常处理流程**，这个会直接失败，因为 `__alltrap` 指令是存放在内核态的 `.text` 段中，CPU 取值会尝试使用 VA 查询 PA，而此时用户态的页表根本查不到这个 VA 相关的信息。，

一、为什么必须有 trampoline？

RISC-V 架构下，用户态和内核态切换（系统调用、异常 / 中断）有两个硬件限制，直接切换会失败，必须靠 `trampoline` 过渡：

1. **特权级与页表切换的“顺序矛盾”**：`RV39` 模式下，内存空间被分为两个不连续的虚拟内存空间，分别存储了用户态的数据和内核态的数据：`user_satp` 只知道用户态的页表地址；`kernel_satp` 只知道内核态的页表地址；
 1. 如果我们先修改 `satp` 再调用 `ecall`：此时修改 `satp` 会直接异常，因为用户态不允许修改 `satp` 页表；
 2. 如果我们先调用 `ecall` 再修改 `satp`，我们的执行是这样的：
 1. 执行 `ecall` 陷入内核态，此时 `satp` 仍然指向 `user_satp`；
 2. `ecall` 或异常触发后，CPU 会自动跳转到 `stvec` 寄存器指向的地址（也就是 `__alltrap` 函数的 VA）。然而，`__alltrap` 指令是存放在内核态的 `.text` 段中，CPU 取值会尝试使用 VA 查询 PA，而此时用户态的页表根本查不到这个 VA 相关的信息。
2. 切换时内核态数据/代码的可访问性：在任意 trap（主动 `ecall` 触发或被动触发）发生时：
 1. CPU 会提升到 S 模式，此时 `satp` 还指向 `user_satp`；
 2. CPU 尝试从 `stvec` 加载异常处理的代码，这会直接异常，因为 `stvec` 指向的代码是在内核态的

`.text` 中，用户态页表根本无法访问。

应用栈和应用内核栈以及内核栈

应用的内核栈，是操作系统为每个应用程序（进程）单独分配的、仅在应用“陷入内核态”时使用的专属栈——本质是一块内核地址空间中的物理内存，专门用于承载应用在核内态运行时的函数调用、局部变量、上下文数据（比如系统调用处理、异常处理的栈帧）。

简单说：应用在用户态（比如运行自己的代码）时，用的是「用户栈」；一旦触发系统调用、异常 / 中断（比如 `ecall` 指令、页错误），切换到内核态后，内核会自动切换栈指针（`sp`）到该应用的「内核栈」，所有内核代码（如 `trap_handler`、系统调用处理函数）都在这个栈上运行。

而内核自身还有一个“内核主栈”（Kernel Main Stack），与所有应用的内核栈独立：

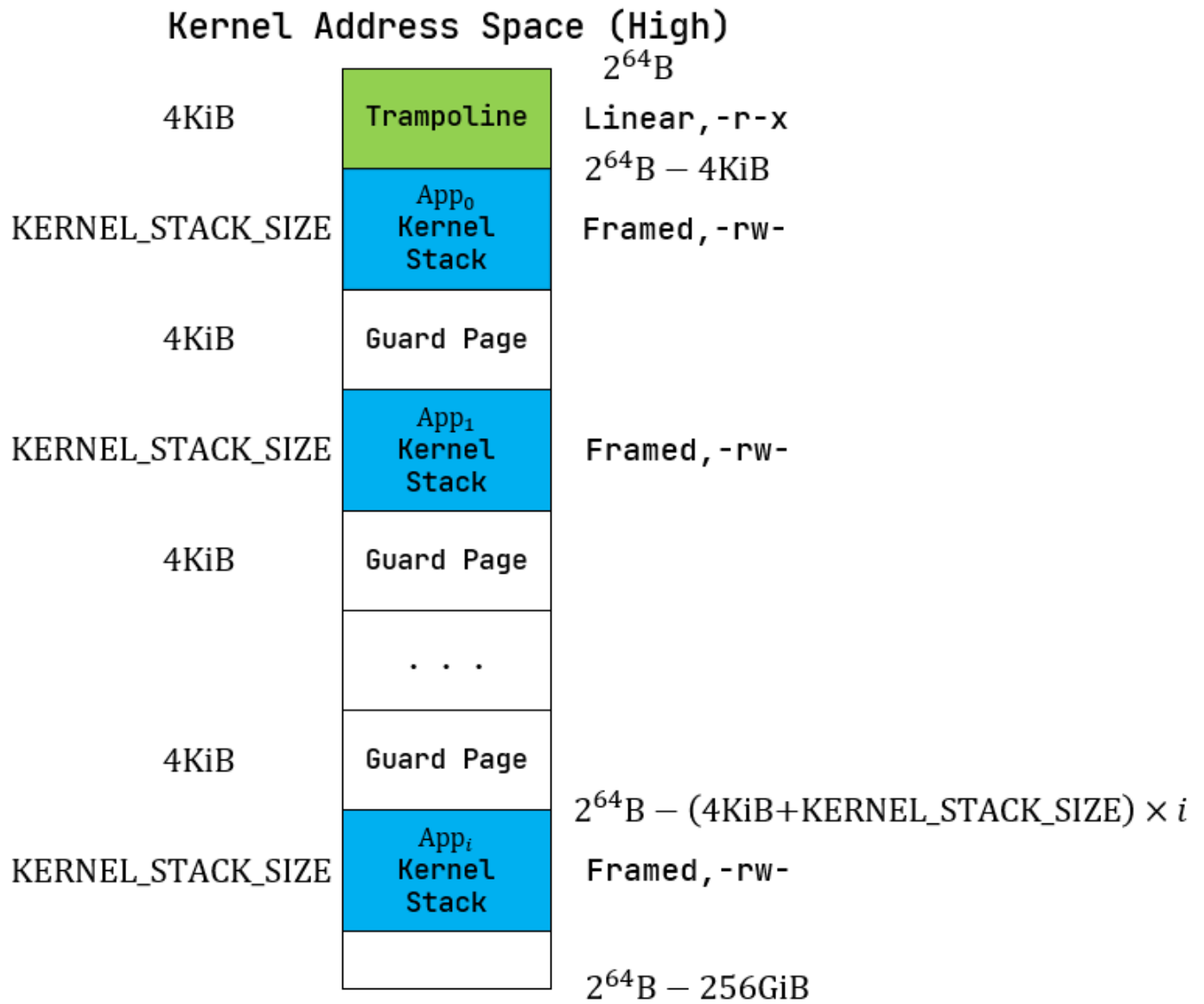
- 内核主栈：用于内核初始化（如 `main` 函数）、硬件中断处理（与应用无关的中断，如时钟中断）等内核原生流程；
- 应用的内核栈：仅用于“该应用触发的内核态操作”（如该应用的系统调用、该应用导致的页错误）；
- 两者物理页独立、地址空间独立，互不干扰。

SV39模式下的地址空间

SV39 模式遵循符号扩展（Sign Extension）规则，所以地址空间的 `[63:39]` 必须和第 38 位保持一致，否则无法通过 MMU 的校验，也就是说，地址空间被分为了 `[0, 0x3F FFFF FFFF FFFF)` 和 `[0xFFFF FFC0 0000 0000, 0xFFFF FFFF FFFF FFFF)` 两个不连续的 256GiB 的地址空间。

在 `rCore` 中，我们一般把高256GiB的内存空间用来存储 `trampoline` 和应用内核栈，按照如下格式：

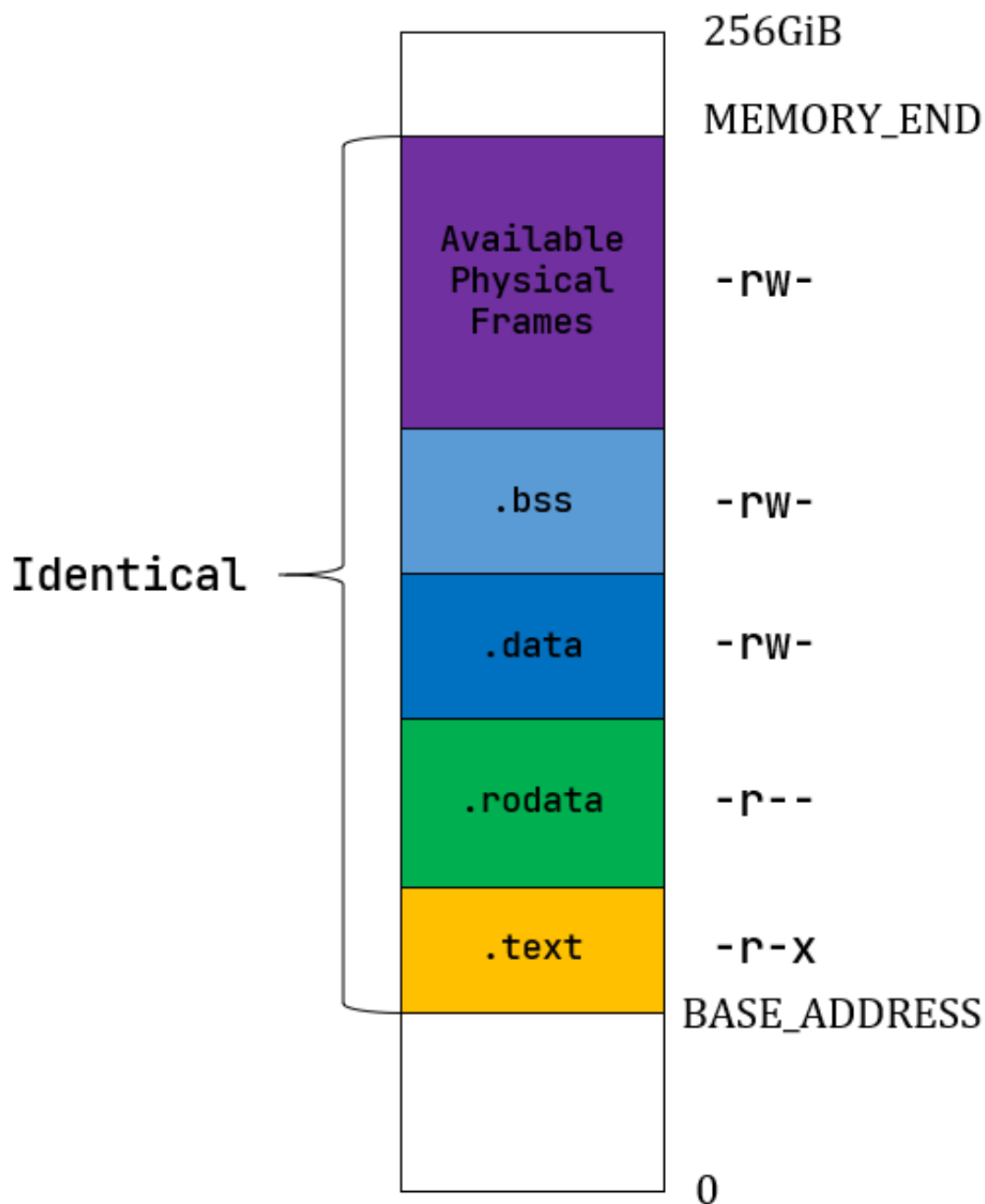
1. `trampoline` 4KiB；
2. `app 0 kernel stack` 根据 `config#KERNEL_STACK_SIZE` 定义；
3. `Guard Page`；
4. `app 1 kernel stack`；
5. `Guard Page`；
6. ...



低256GiB，我们用来存储内核的段，以及空闲物理页帧（Available Physical Frames），按照如下格式：

1. Available Physical Frames
2. `.bss`
3. `.data`
4. `.rodata`
5. `.text`

Kernel Address Space (Low)



1. 这里在初始化的过程中，通过恒等映射将 `.bss/.data/.rodata/.text` 段映射到了物理内存，这也是我们在 `linker.ld` 内声明的。此外，我们的项目是基于qemu模拟的，而qemu的内存空间从 `0x80000000` 开始，也就是说如果我们在linker里设置了错误的物理内存地址，例如 `0x78000000`，那么程序会直接启动失败。
2. 代码段的 `u` 都设置为0，也就是只能在内核态访问；
3. `.text` 不允许被修改；
4. `.rodata` 不允许被修改，也不允许从它上面取指执行；
5. `.data/.bss` 均允许被读写，但是不允许从它上面取指执行

虚拟内存到物理内存

内核态的分页逻辑

开启分页后，CPU 不会区分“内核态地址”和“用户态地址”——所有访存地址都会被视为虚拟地址，必须经过 MMU 查页表转换。哪怕是内核访问自己的代码、数据，也得通过映射才能找到对应的物理页。

举个例子，在我们的 `rCore` 实现中，我们在链接器中设置的起始地址是 `0x80200000`，我们的访问是这样的：

1. 内核代码段存放在物理地址 `0x80200000`（`ekernel` 之前）；
2. 启用分页前，CPU 执行 `lw x1, 0x80200000`，直接访问物理内存 `0x80200000`；
3. 启用分页后，CPU 会把 `0x80200000` 当成虚拟地址：
 1. 若页表中没有“虚拟地址 `0x80200000` → 物理地址 `0x80200000`”的映射，MMU 会触发页错误，内核直接崩溃；
 2. 只有建立了映射，MMU 才会把虚拟地址转成物理地址，内核才能正常执行代码、访问数据。

内核态的内存（代码、数据、栈、页表本身），是“需要访问的物理页”中最核心的一部分，必须优先建立映射——否则内核自己都跑不起来。

内核态需要映射的物理页具体有哪些

1. 内核自身的代码段（`.text`）、数据段（`.data`）、`.bss` 段：内核的指令、全局变量、未初始化变量都存在这些物理页中，是内核运行的基础——必须建立映射，否则内核无法执行指令、访问全局变量。
2. 内核栈（Kernel Stack）：内核线程 / 进程的栈空间（用于函数调用、局部变量存储），本质是一块物理页——必须建立映射，否则内核函数调用会栈溢出或访问非法地址。
3. 页表页（根页表、中间级页表）：页表本身也是物理页，内核需要修改页表项（如为用户进程建立映射），就必须访问这些物理页——因此页表页自身也需要建立映射（通常是恒等映射）。
4. 内核管理的空闲物理页（按需映射）：内核需要分配 / 回收物理页（如用户进程 `malloc` 时），就必须访问这些空闲物理页——通过恒等映射，内核可以直接用“物理地址作为虚拟地址”访问它们，无需额外计算偏移。

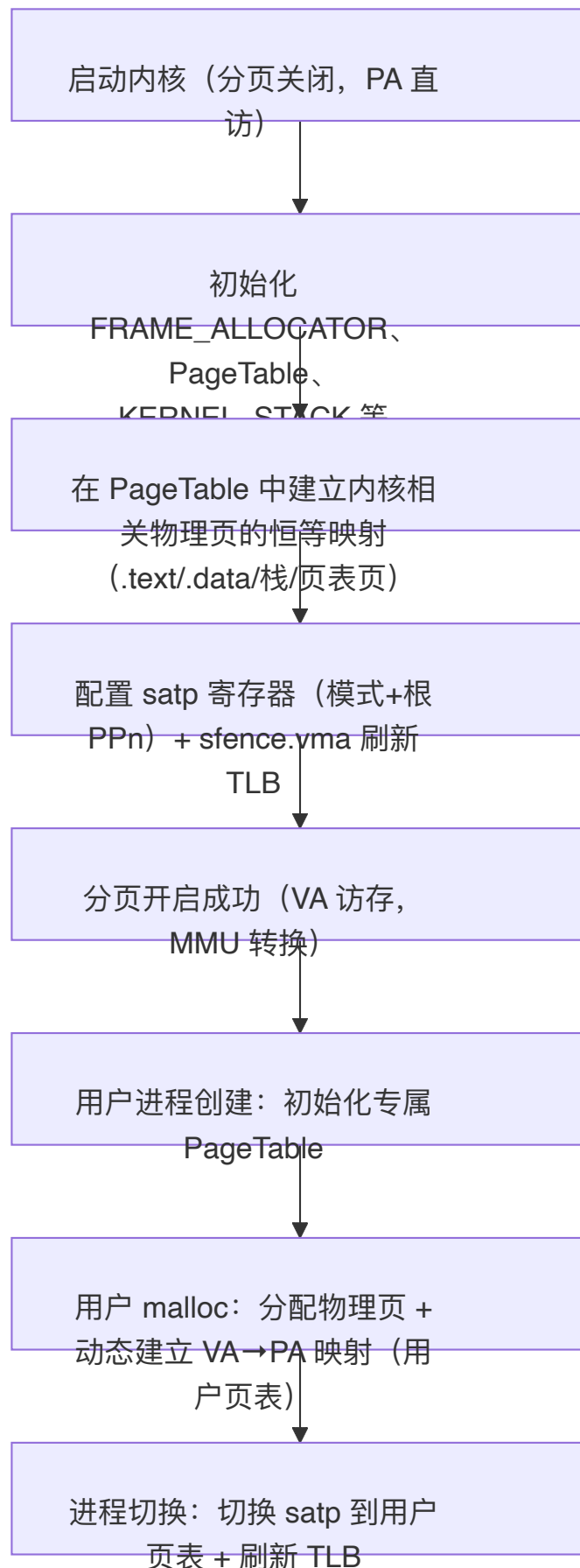
这些映射通常在内核初始化阶段、启用分页之前就建立完成，确保启用分页后内核能正常运行。

如何建立映射

除了“用户态私有数据页”和“按需动态映射的空闲物理页”，内核态所有需要访问的物理页（代码段、数据段、`.bss`、内核栈、页表页），均使用恒等映射（`VA=PA`）。

- 以下场景必须使用 `恒等映射`，因为必须直接访问物理页，且地址不能变：
 - 内核代码段（`.text`）、数据段（`.data`）、`.bss` 段：链接器指定的地址就是物理地址，内核代码 / 全局变量的引用直接依赖这个地址，改用非恒等映射会导致“代码指令地址找不到”“全局变量访问错位”，内核无法运行；
 - 内核栈（Kernel Stack）：栈指针（`sp`）指向的是物理地址，函数调用、局部变量存储依赖这个地址的连续性，非恒等映射会导致栈访问异常（如栈溢出、数据错乱）；
 - 页表页（根页表、中间级页表）：页表项（PTE）中存储的是物理页号（PPN），内核修改页表时需要直接访问页表页的物理地址，恒等映射能让内核“以物理地址为虚拟地址”直接操作，无需额外计算偏移。
- 空闲物理页是每次用户态程序申请内存时，从 `FRAME_ALLOCATOR` 中得到的随机映射。

我们整体的流程可以总结为：



PhysPageNum和FrameTracker

使用 `FrameTracker` 代替 `PhysPageNum` 这个是基于 `RAII` 的思想的设计：我们为 `FrameTracker` 实现了 `Drop`，这样我们就不用手动的在物理栈帧退出作用域时手动的 `dealloc()` 了。

而这里，我们之所以使用 `FrameTracker` 来实现，而不是直接为 `PhysPageNum` 实现 `Drop` 接口，是因为两者的语义是完全不一样的：

- 1. `PhysPageNum` 是物理页帧的编号，他是一个无所有权的“标识”；
- 2. `FrameTracker` 是物理页帧的“资源句柄”，是 **所有权的管理者**。

类型	核心语义	角色定位	关键特性
PhysPageNum	物理页帧的“编号”（如 0x100、0x200）	无所有权的“标识 / 索引”	1. 仅存储页号，不关联“分配状态”； 2. 可复制、可传递，不影响物理页； 3. 无法感知“是否被分配”。
FrameTracker	物理页帧的“资源句柄”	有所有权的“管理者”	1. 持有 PhysPageNum，代表“独占该物理页”； 2. 实现 Drop，离开作用域自动调用 dealloc； 3. 不可随意复制（通常 !Clone），确保所有权唯一。

类似于房间，`PhysPageNum` 是房间号，而 `FrameTracker` 是业主。

再从两个struct的实现来看：

- 1. `PhysPageNum` 是简单的包装类型（`struct PhysPageNum(pub usize)`），可复制，可拷贝等；
- 2. `FrameTracker` 不可复制，不可拷贝；

```
1  /// physical page number
2  #[derive(Copy, Clone, Ord, PartialOrd, Eq, PartialEq)]
3  pub struct PhysPageNum(pub usize);
4
5
6  /// tracker for physical page frame allocation and deallocation
7  pub struct FrameTracker {
8      /// physical page number
9      pub ppn: PhysPageNum,
10 }
```

内存的内核空间 and 用户空间

在我们使用内存的过程中，有两个一直被提到的名词：`内核空间` 和 `用户空间`，那硬件和操作系统是怎么区分内核空间和用户空间的呢？

首先需要知道的是，这两个空间都是操作系统的概念。硬件（CPU/MMU）对物理内存的所有页帧一视同仁。所谓“内核物理内存”“用户物理内存”，是操作系统为了管理方便（避免冲突、统一回收）而做的软件划分（比如把物理地址 0x80000000~0xFFFF0000 划分为内核专用），硬件不感知这个划分。

硬件判断“当前访问的是内核空间还是用户空间”，核心依据是 虚拟地址（VA），而非物理地址（PA）—— 具体是“VA 高位特征 + 页表项 U 位 + 当前 CPU 特权级”三者协同：

- VA 高位：初步归类（例如如 Sv39 的 0x0000_0000_0000~0x7FFF_FFFF_FFFF 为用户 VA，0x8000_0000_0000~0xFFFF_FFFF_FFFF 为内核 VA）；
- 页表项 U 位：精准权限（`U=1` → 允许用户态访问，`U=0` → 仅允许内核态访问）；
- CPU 特权级：最终校验（用户态不能访问 U=0 的 VA，内核态可访问所有 VA）。

总结来说：

- 硬件：只认“VA 高位 + PTE U 位 + 特权级”，不管物理内存的软件划分；
- 操作系统：制定 VA 划分规则、设置 PTE 权限、管理 PA 分配，让硬件的判断逻辑服务于“内核 / 用户隔离”；

而在 `rCore` 的开发过程中：这个特权级切换路径是：M 模式（引导）→ S 模式（内核运行）→ U 模式（用户进程运行）

我们通常是在 M 模式下，开始加载一段内核引导代码：

1. 通常我们会在链接器里声明一个 `ekernel` 表示内核代码的结束地址，并且进行内存对齐；
2. 内核会维护一个 memory allocator，这个 memory allocator 会使用 `ekernel` 物理内存作为分配的起始点，也就是用户态内存的起始点，并在内核代码的配置中声明一个值作为用户态地址的终点。

随后，内核在分配内存的过程中就围绕这一片被划分到用户态的虚拟内存来分配对应的物理内存，同时维护页表项。也就是说，我们现在已经满足了 VA 高位 + PTE U 位这两个条件。

而特权级这个条件，我们是基于 trap 来实现的，在用户态下，如果需要访问内核空间代码，则需要通过 trap 跳转到对应的特权级。

from 和 into

在 Rust 中，From 和 Into 是一对类型转换相关的 trait，核心作用是定义“安全、无歧义的类型转换规则”，避免手动写重复的转换代码，同时让代码更简洁、可读性更强。二者本质是“同一转换的正反两面”，遵循“实现 `From<A>` for B，自动获得 `Into<B>` for A”的设计（无需重复实现）。

一般而言，当我们为类型 U 实现了 From Trait 之后，可以使用 `U::from(_: T)` 来从一个 T 类型的实例来构造一个 U 类型的实例；而当我们为类型 U 实现了 Into Trait 之后，对于一个 U 类型的实例 u，可以使用 `u.into()` 来将其转化为一个类型为 T 的实例。

SV39 多级页表的硬件基址

- `SATP` Supervisor Address Translation and Protection，直接控制处理器是否启用虚拟地址转换（MMU 功能），并指定页表的物理地址。

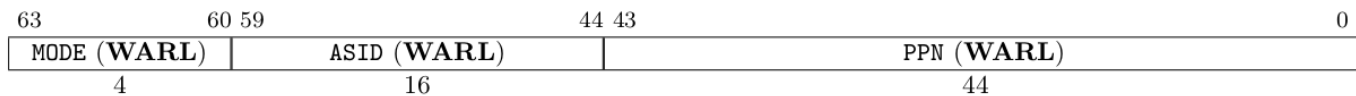
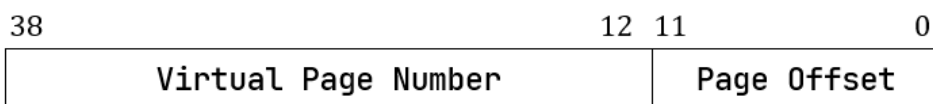


Figure 4.14: RV64 Supervisor address translation and protection register `satp`, for MODE values Bare, Sv39, and Sv48.

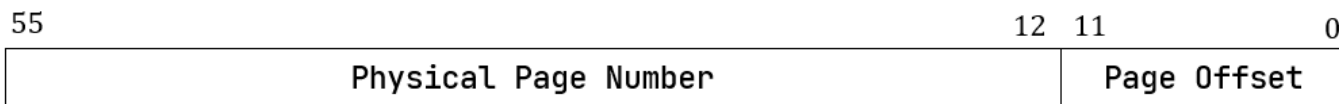
- `MODE` 控制 CPU 使用哪种页表实现；
- `ASID` 表示地址空间标识符，这里还没有涉及到进程的概念，我们不需要管这个地方；
- `PPN(Pyhsical Page Number)` 存的是根页表所在的物理页号。这样，给定一个虚拟页号，CPU 就可以从三级页表的根页表开始一步步的将其映射到一个物理页号。
- 当 `MODE` 设置为 0 的时候，代表所有访存都被视为物理地址；
- 而设置为 8 的时候，SV39 分页机制被启用，所有 S/U 特权级的访存被视为一个 39 位的虚拟地址，它们需要先经过 MMU 的地址转换流程，如果顺利的话，则会变成一个 56 位的物理地址来访问物理内存；否则则会触发异常，这体现了分页机制的内存保护能力。

地址格式与组成

Virtual Address (39bits)



Physical Address (56bits)



在 `sv39` 下，我们的 VA 总共有 39 位。并且分为两个部分：

- `PO(Page Offset)` 用于表示页内偏移量，这个和我们页的大小有关，假设每页是 `4KB`，那么就需要 `12` 位；
- `VPN(Virtual Page Number)` 用于 PPN，在映射完之后通常被扩展，例如可能从 `27` 位扩展到 `44` 位。

这里需要注意的是，我们的 PPN 的范围是要大于 VPN 的，核心是为了“解耦虚拟空间需求和物理空间能力”：

1. 虚拟空间只需要满足应用的“观感需求”：应用需要的是“连续、足够大的地址空间”，而非“实际的物理内存”，而 512GB 虚拟空间足够覆盖绝大多数场景
2. 物理空间需要满足硬件的“扩展需求”。

而解耦的核心价值在于：

- 应用不依赖物理内存大小：即使物理内存只有 8GB，应用也能使用 512GB 虚拟空间（通过“按需分页”和“交换分区”，裸机场景可省略交换分区）；
- 物理内存可灵活分配：多个应用的虚拟空间可以映射到物理内存的不同区域，物理内存不足时，内核可以回收不常用的物理页（换出到磁盘），再分配给需要的应用；
- 硬件升级无需修改应用：当物理内存从 8GB 升级到 64GB，应用无需重新编译——内核只需更新页表映射，应用的虚拟地址空间完全不变。

页表项的数据结构抽象与类型定义

63	54 53	28 27	19 18	10 9	8	7	6	5	4	3	2	1	0
<i>Reserved</i>	PPN[2]	PPN[1]	PPN[0]	RSW	D	A	G	U	X	W	R	V	
10	26	9	9	2	1	1	1	1	1	1	1	1	

- SV39是一个三级页表，其中 [53: 10] 这 44 位是物理页号，并且对应了在TLB中的一级索引，二级索引一级三级索引；
- 低8位是对应的物理页的标志位：
 - V 对应物理页是否有效；
 - R, W, X 表示物理页的读，写，执行；
 - U 代表了在 User Mode 下是否有权限执行；
 - G
 - A 代表了页面代表对应的虚拟页面是否被访问过，是 [全局物理内存的页置换算法](#) 的核心数据；
 - D 代表了页面代表对应的物理页面是否被修改过，用于判断缓存是否可用。此外，它可以协助我们判断 [优化页面换出](#)。

全局物理内存的页置换算法

“全局物理内存的页置换算法（如clock，二次机会算法）”是缓存淘汰的核心逻辑，具体规则可能如下：

- 内核扫描页表，检查 A 位；
- 所有 A 为0的页面最近未被访问过，直接淘汰；
- 如果 A 为1，则页面最近被访问过，我们此时将其设置为0；
- 再回到 <1>，如果 A 为0，则近期末访问，进行淘汰；

核心逻辑在于，A 的状态为 1 只是代表过去有被访问过，所以我们每次都需要重置，这样在下次扫描的时候我们就知道最近有没有被访问过。

优化页面换出

优化页面置换

D 位的核心价值是 减少磁盘 I/O 开销（这是操作系统中最昂贵的操作之一）：

当我们在进行内存操作时，并不会立即数据全部写入到硬盘（或其他I/O），而是先修改内存，并且将页表中的 D 设置为1。

而当我们需要进行页面置换，或者 flush 之类的操作将数据更新时，D 位可以帮我们判断，当前内存页是可以直接释放还是需要刷新到硬盘！

实现COW

假设父进程 P 创建子进程 C，共享某物理页 PhysPage(0x100)，页表项初始状态：V=1, R=1, W=0, U=1, A=0, D=0（有效、可读、禁止写、用户态可访问、未访问、未修改）。

- 进程创建阶段：共享物理页 + 权限初始化

- 父进程 P 的页表：虚拟页 VPN=0x200 → 物理页 PPN=0x100 (W=0) ；
- 子进程 C 的页表：相同虚拟页 VPN=0x200 → 相同物理页 PPN=0x100 (W=0) ；
- 内核维护“物理页引用计数”：PhysPage(0x100) 的引用计数 = 2 (P 和 C 共享) ；
- 关键：父子进程的页表项都设为 W=0 (禁止写)、D=0 (初始无修改)，确保写操作触发异常。

2. 写操作触发异常：进入 Trap Handler

- 假设子进程 C 尝试写 VPN=0x200 对应的虚拟页；
- 处理器检查页表项：V=1 (有效) 但 W=0 (禁止写) → 触发写保护异常 (Store Page Fault) ；
- RISC-V 会将异常原因 (cause=0x24, Store Page Fault)、触发写操作的虚拟地址 (stval=VPN=0x200) 存入 CSR 寄存器，跳转到内核的 Trap Handler。

3. Trap Handler 处理逻辑 (核心步骤)：Handler 需完成“识别 COW 页 → 复制物理页 → 更新页表 → 恢复执行”，具体：

1. 识别当前页是 COW 共享页

- 从 stval 取出触发异常的虚拟地址 VA，找到子进程 C 对应页表项；
- 检查页表项特征：V=1, W=0, D=0 + 物理页 PPN=0x100 的引用计数 > 1 → 确认是 COW 共享页 (非普通写保护页)。

2. 分配新物理页并复制数据

- 内核从空闲物理页池分配新页 PhysPage(0x101)；
- 将旧页 PhysPage(0x100) 的所有数据复制到新页 PhysPage(0x101) (仅复制一次，完成后两页数据完全一致)。

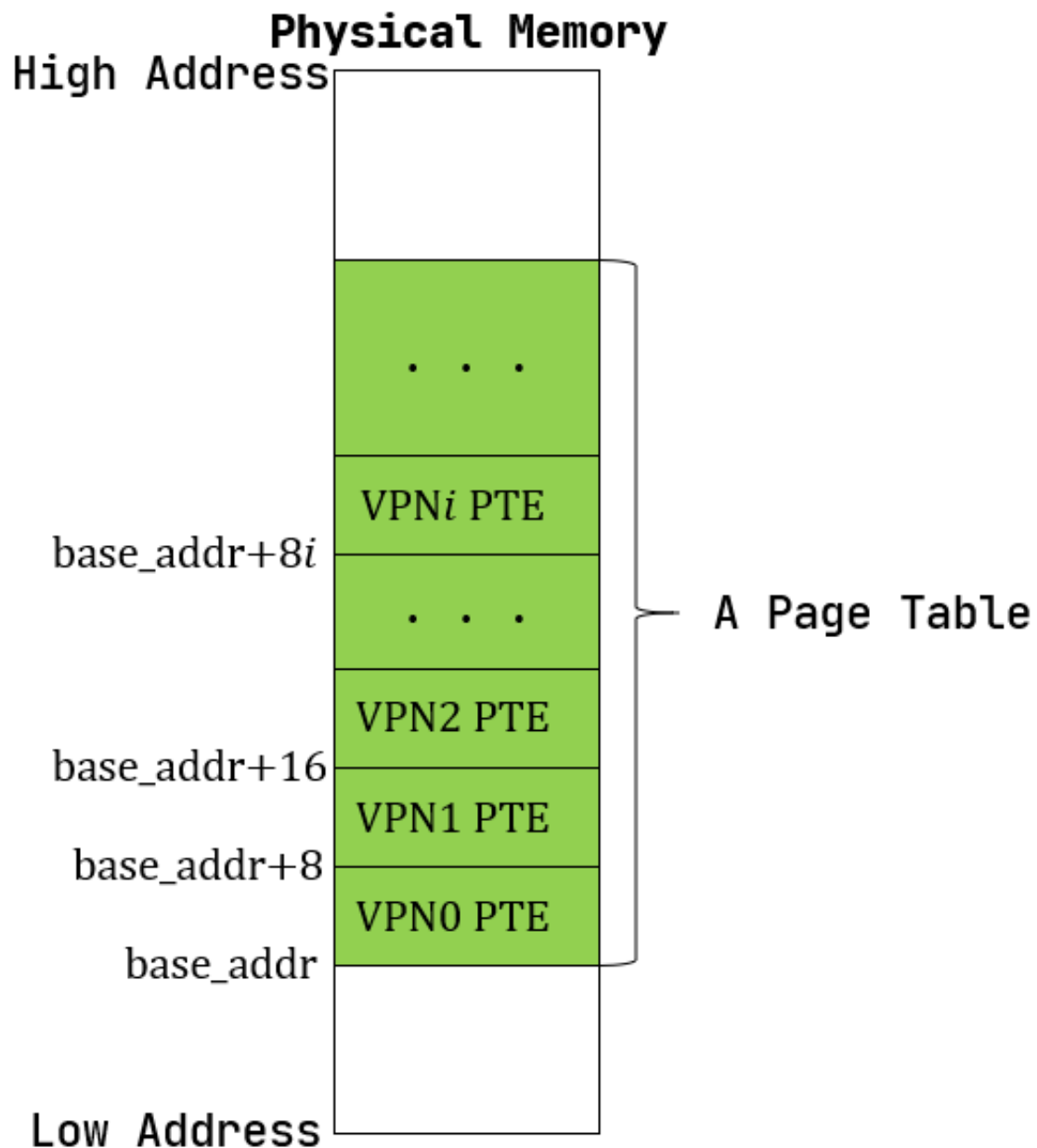
3. 更新页表项权限和标志位，这里需要注意的是，现在是内核在进行内存页复制，复制前后的两个内存页的 **D = 0**，因为此时仅仅是处理 **Storage Page Fault**，真正的页面操作不在这个异常处理的范畴中。

4. 刷新 TLB (避免旧映射干扰)

4. 恢复进程执行，此时写入操作完成，将页面 **D = 1**。

多级页表

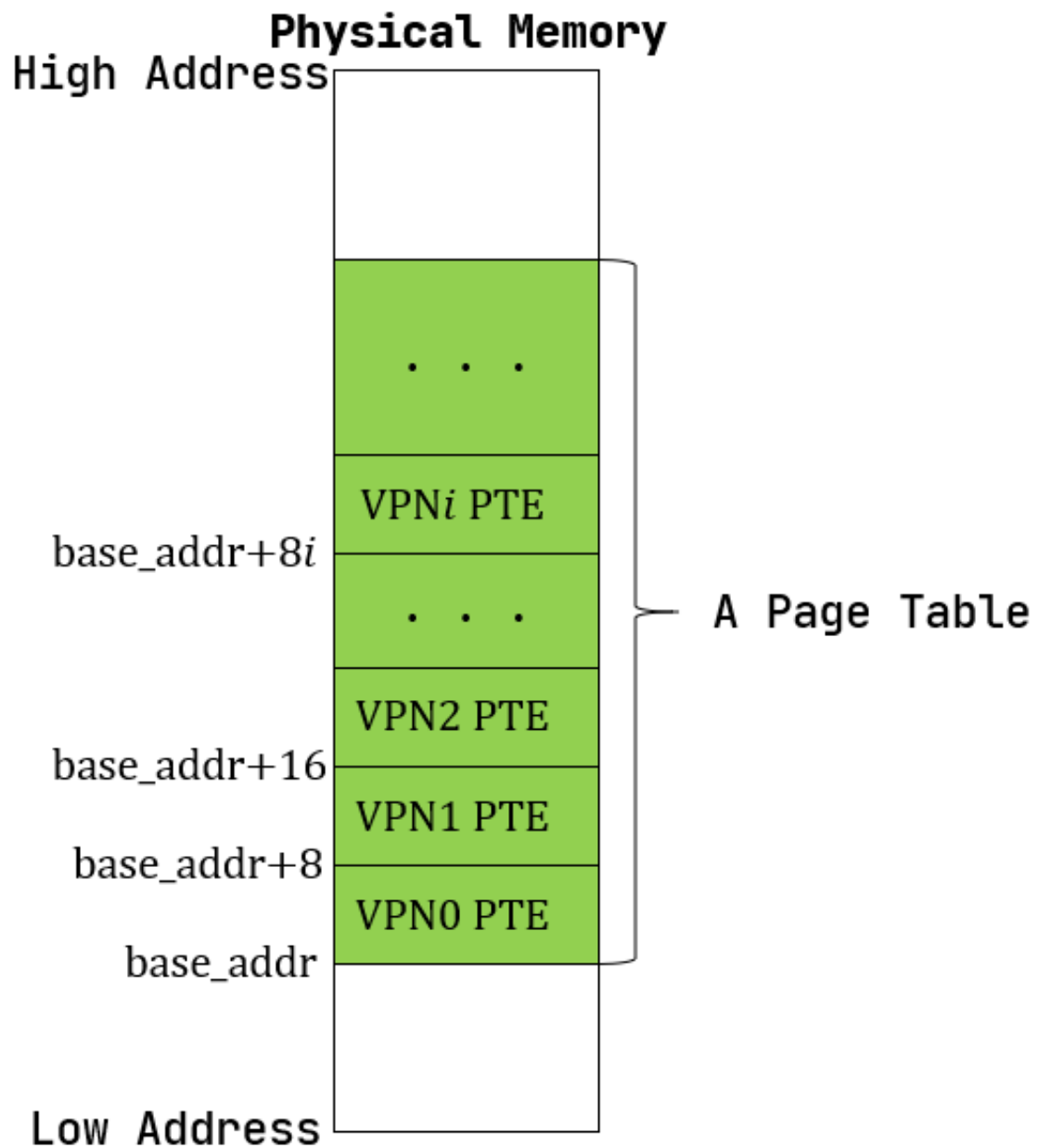
单级页表/线性表



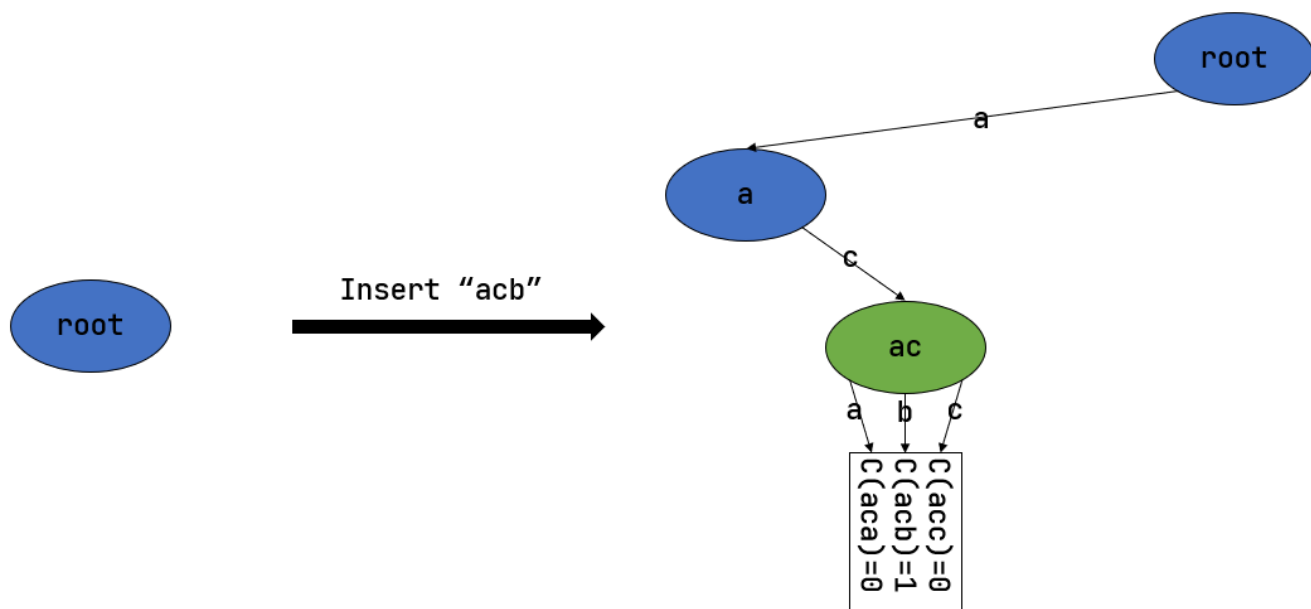
最简单的页表实现，就是将全部的VPN -> PPN的映射关系全部存储到内存中，按照我们目前的RV39中页大小4KB，我们总共需要 $2^{27} * 8$ 也就是 1GB。并且，每个进程都必须有一个。这是完全不可接受的。为此，我们需要引入一个多级页表。

字典树

假设存在一个输入字符集为 $x = \{a, b, c\}$ ，长度为 n 的字符集，那么它的结构可能如下所示



可以看到，上面是一颗完整的字典树，而在我们构造这个字段树的过程中，最开始整棵树是空的。只有当我们写入到一个特定的字符时才会有新的节点生成。



可以看到，我们此时只有一个节点。

事实上 SV39 分页机制等价于一颗字典树。27 位的 VPN 可以看作长度为 3 的三个字符串，字符集为 $x = \{0, 1, 2, \dots, 511\}$

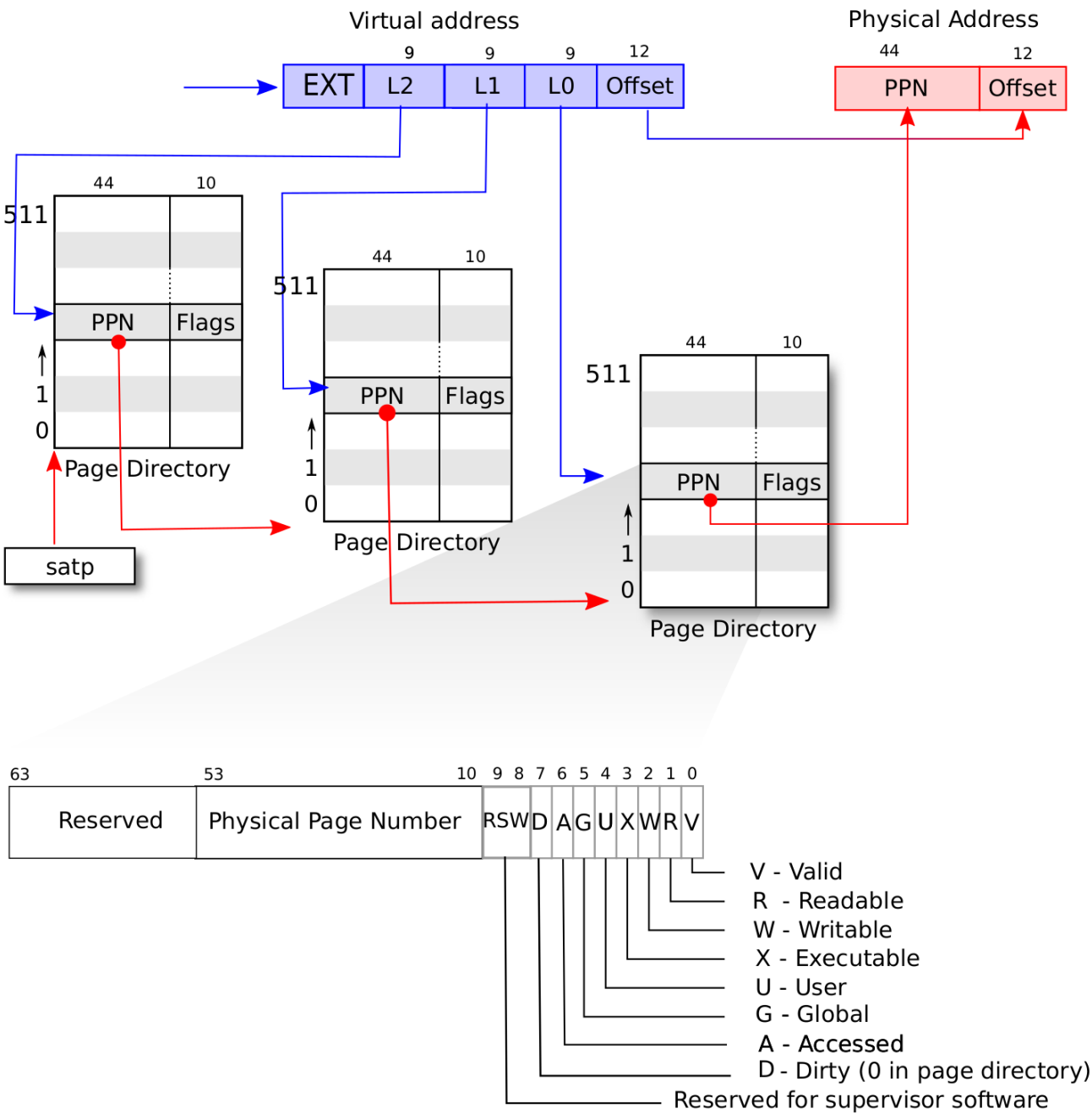
大表

RV39 支持大表查找和访问，具体逻辑如下

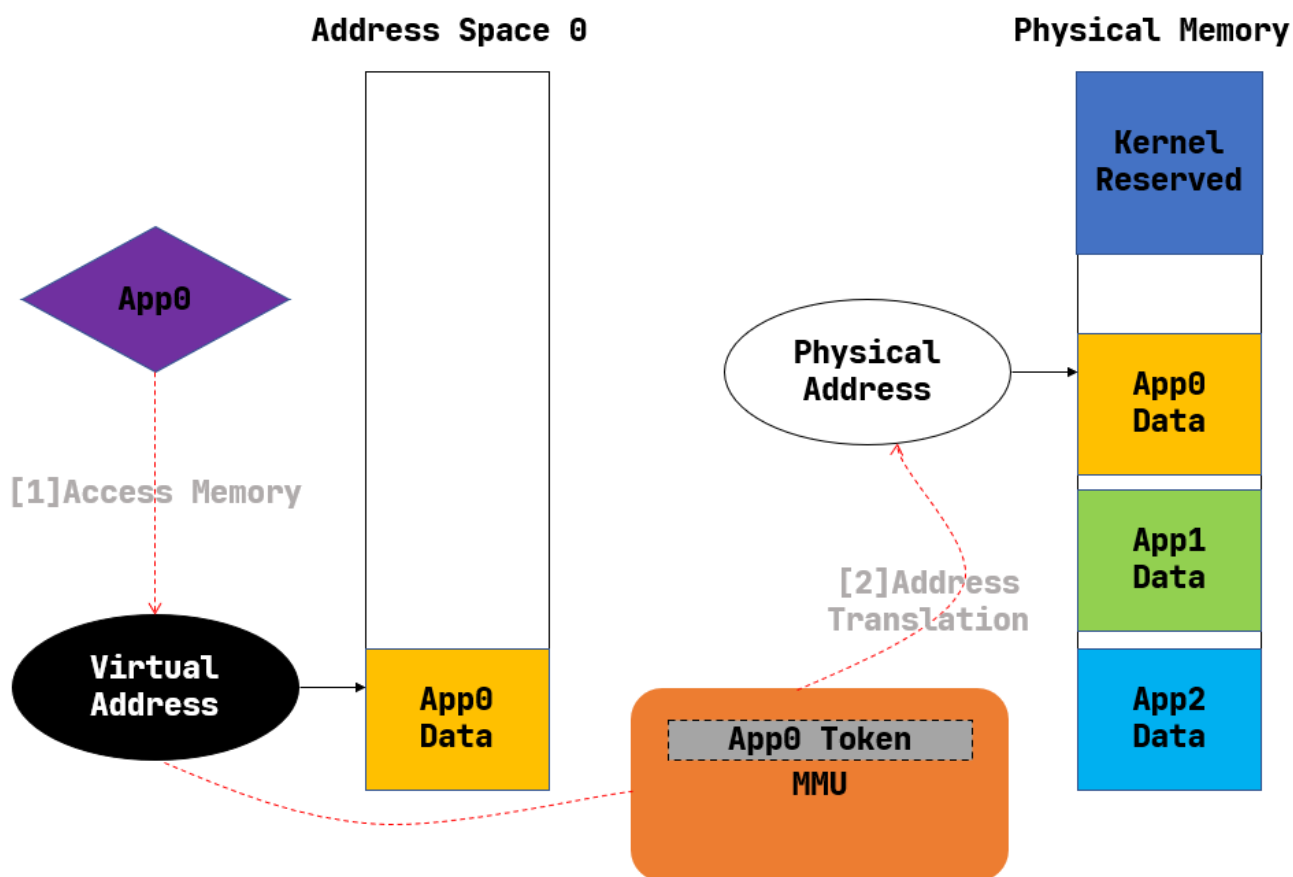
- 虚拟地址 (VA) 结构：39 位有效位，拆分为 4 部分：
 - 高位 2 位：保留（必须为 0，否则为非法地址）；
 - VPN [2]：一级页表索引（9 位，对应一级页表项位置）；
 - VPN [1]：二级页表索引（9 位，对应二级页表项位置）；
 - VPN [0]：三级页表索引（9 位，对应三级页表项位置）；
- 页表项 (PTE) 结构：64 位，低 8 位为标志位 (V/R/W/X/U/A/D/G)，高 56 位为物理页号 (PPN) 或下一级页表的 PPN（因 Sv39 物理地址最大 56 位）。
- 核心标志位 V 的语义
 - V=1：页表项有效 (Valid)，是“合法的页表项”（可能是“指向下级页表”的页目录项，也可能是“指向物理页”的叶节点项，或“大页映射”项）；
 - V=0：页表项无效，访问时触发缺页异常 (Page Fault)

大页级别	叶节点页表项	映射粒度	VA 中 “用于偏移” 的位数	对应范围
4KB 页	三级页项	4KB = 2 ¹²	12 位 (仅 Offset)	单个 4KB 页
2MB 大页	二级页表	2MB = 2 ⁹⁺¹²	9 (VPN [0]) +12=21 位	2MB
1GB 大页	一级页表	1GB = 2 ⁹⁺⁹⁺¹²	9 (VPN [1]) +9 (VPN [0]) +12=30 位	1GB

SV39转换过程

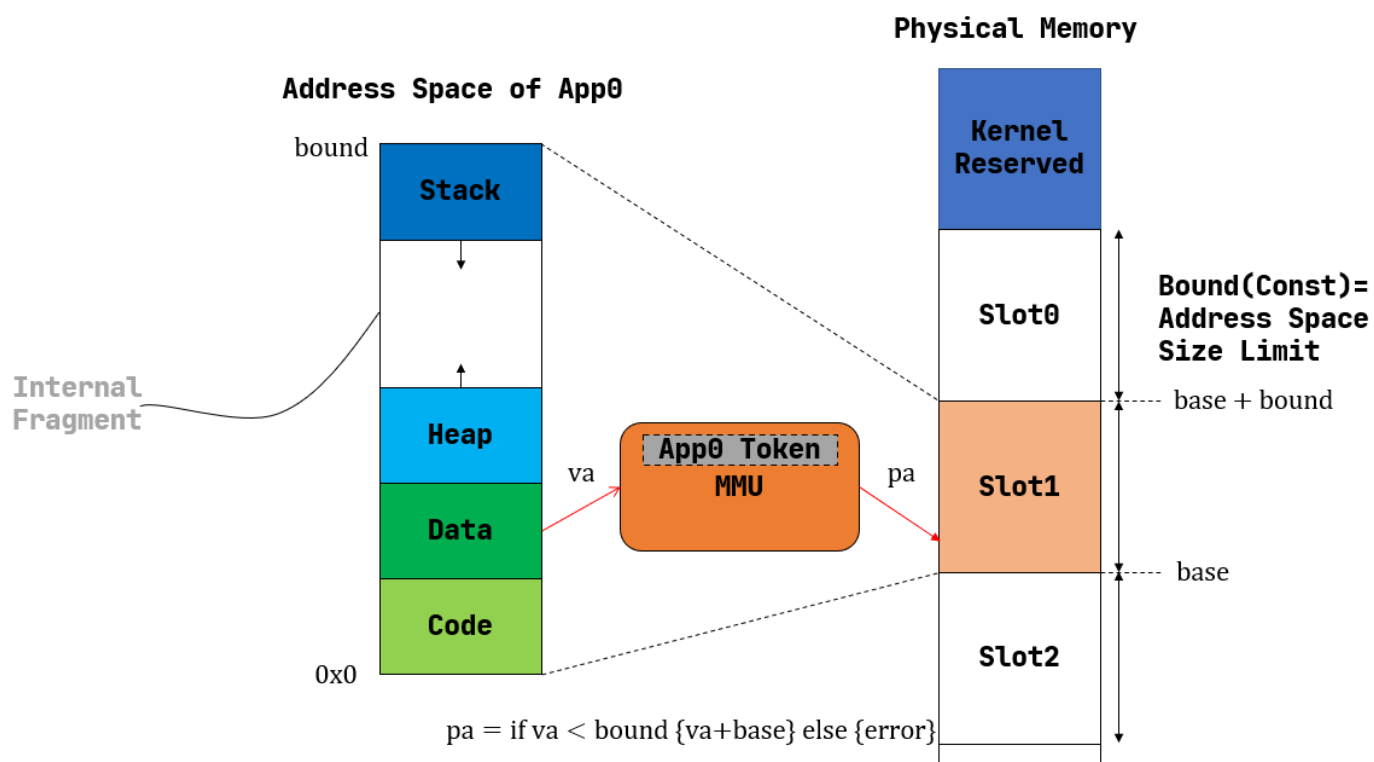


内存管理的一些示例

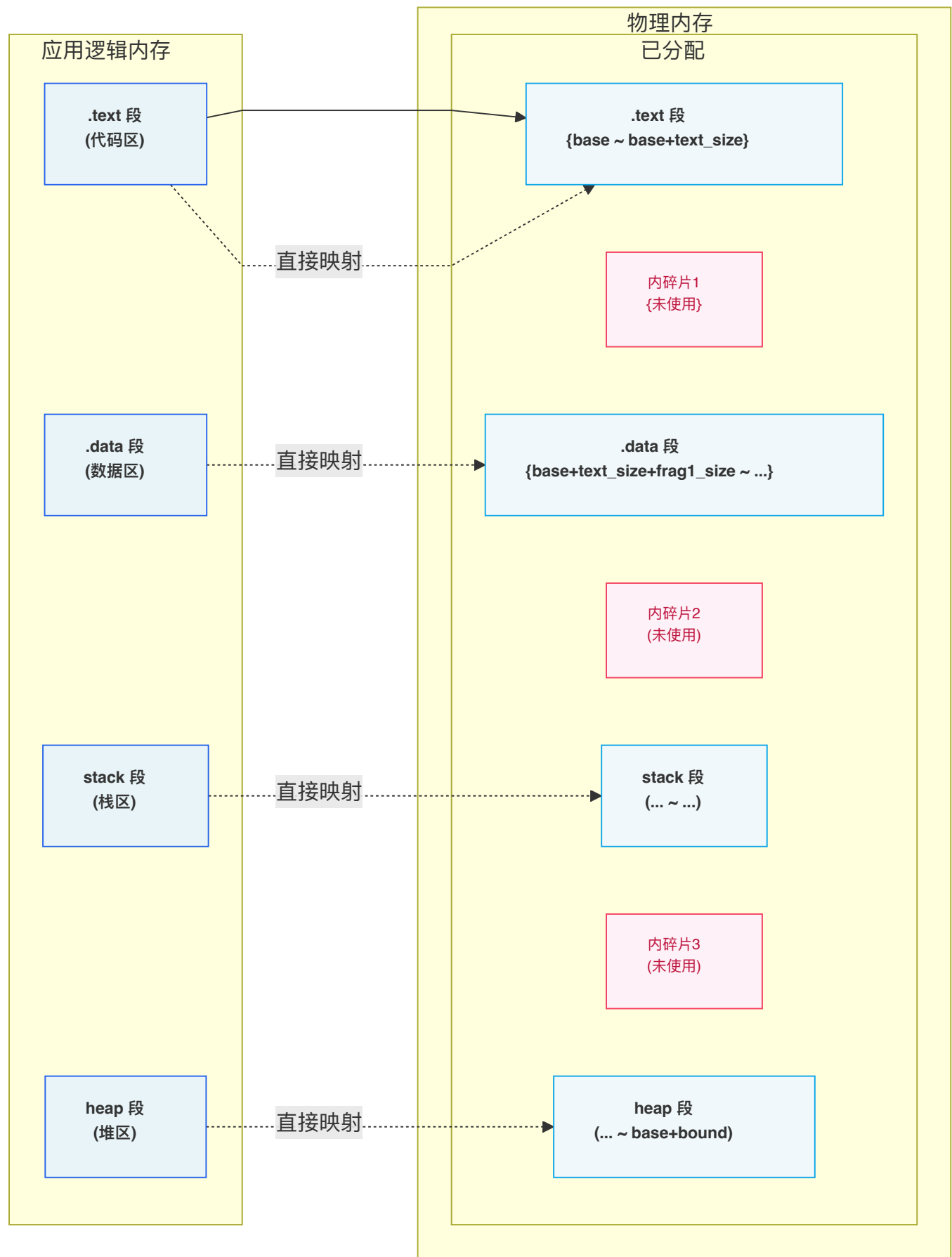


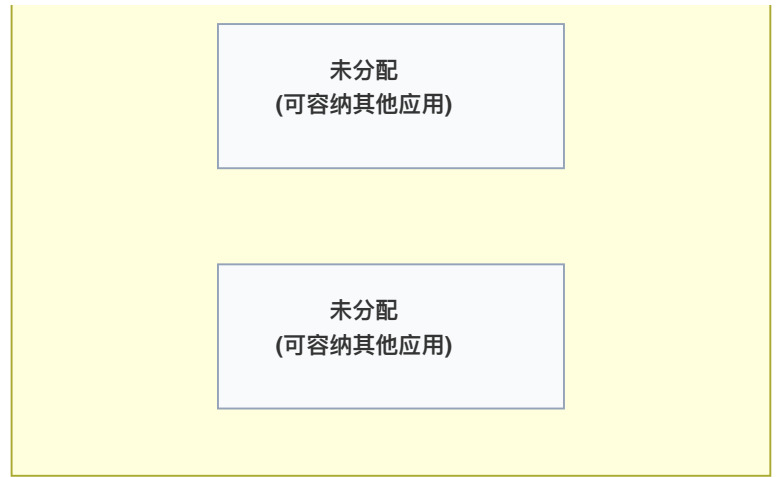
分段内存管理

分段内存管理一：使用固定内存大小及物理SLOT进行映射

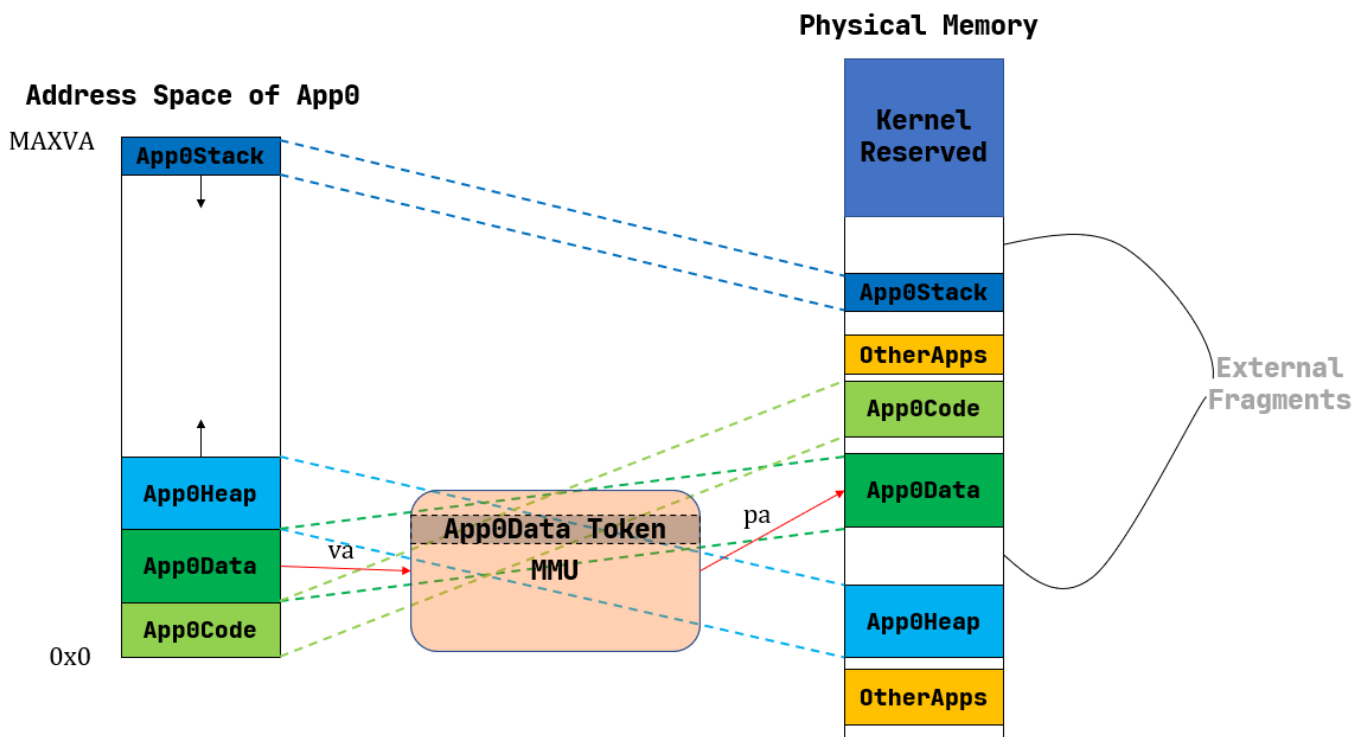


每个应用的地址空间大小限制为一个固定的常数 `bound`，也即每个应用的可用虚拟地址区间均为 `[0, bound)`。随后，就可以以这个大小为单元，将物理内存除了内核预留空间之外的部分划分为若干个大小相同的**插槽 (Slot)**，每个应用的所有数据都被内核放置在其中一个插槽中，对应于物理内存上的一段连续物理地址区间，假设其起始物理地址为 `base`，则由于二者大小相同，这个区间实际为 `[base, base + bound)`。





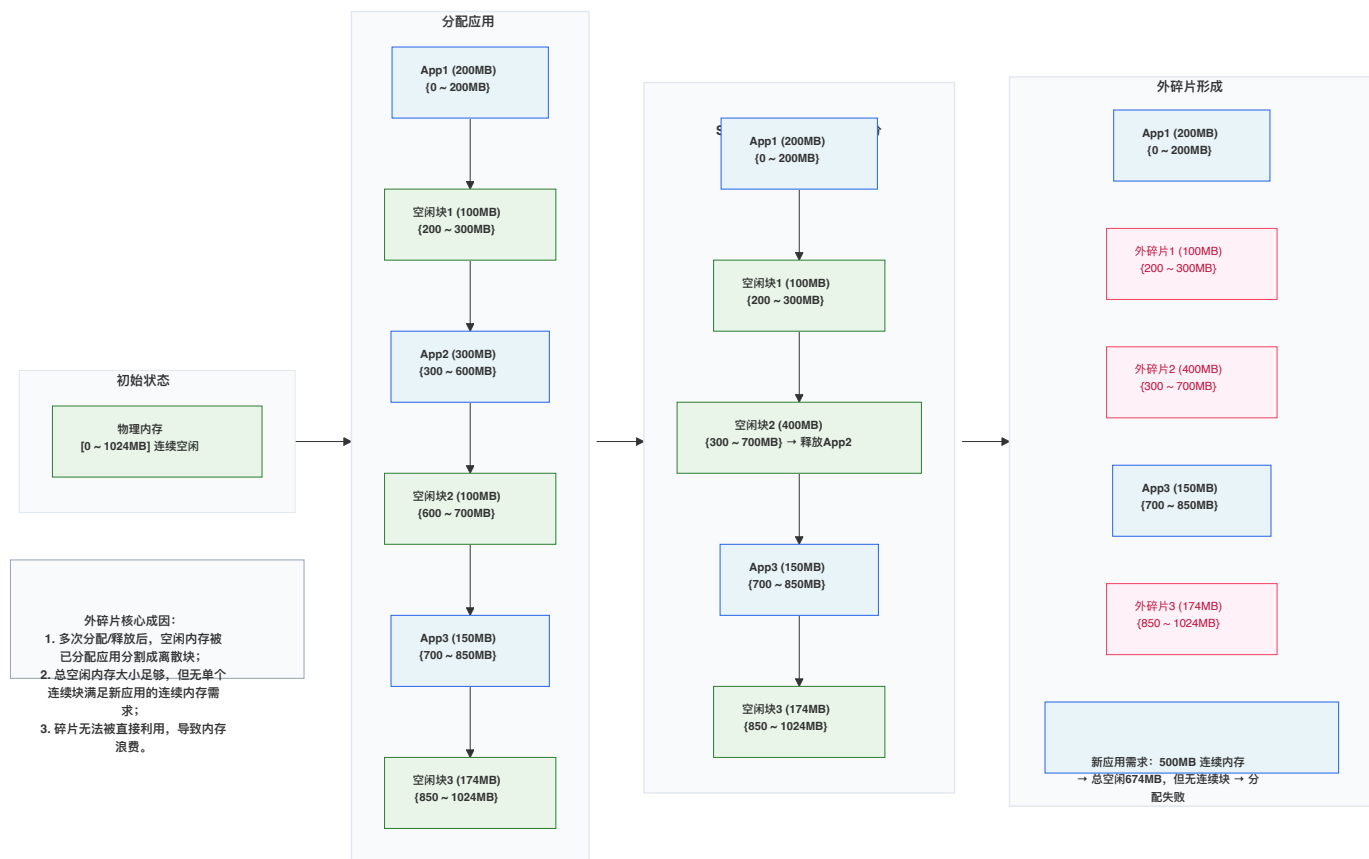
分段内存管理二：使用应用地址空间中的一个逻辑段作为单位来安排应用的数据在物理内存中的布局



相对于分段内存管理一中的内存分配方式，这种方式不是将应用的整个内存空间直接映射到物理内存。而是将应用的物理内存按照 `.text`、`.data`、`stack`、`heap` 等逻辑段按照一定的规则映射到物理内存。

注意到每个段都只会在内存中占据一块与它实际所用到的大小相等的空间。堆的情况可能比较特殊，它的大小可能会在运行时增长，但是那需要应用通过系统调用向内核请求。也就是说这是一种按需分配，而不再是内核在开始时就给每个应用分配一大块很可能用不完的内存。由此，不再有内存碎片了。

尽管内存碎片被消除了，但内存浪费问题并没有完全解决。这是因为每个段的大小都是不同的（它们可能来自不同的应用，功能也不同），内核就需要使用更加通用、也更加复杂的连续内存分配算法来进行内存管理，而不能像之前的插槽那样以一个比特为单位。顾名思义，连续内存分配算法就是每次需要分配一块连续内存来存放一个段的数据。随着一段时间的分配和回收，物理内存还剩下一些相互不连续的较小的可用连续块，其中有一些只是两个已分配内存块之间的很小的间隙，它们自己可能由于空间较小，已经无法被用于分配，这就是 **外碎片 (External Fragment)**。



分页内存管理

之前的两种分段管理的方式:

- 将应用内存看做一个整体: 由于应用内部段的不连续性, 将产生大量的内碎片;
- 将应用内存分为多个逻辑段, 并按段进行映射: 在这种情况下, 在经过多次分配和释放后空闲内存将分散在物理内存中无法统一的使用。例如假设我们存在一个内存块 `[0, 1, 2, 3, 4]`:
 - 将五个内存块都分配, 此时使用的内存块为 `[0, 1, 2, 3, 4]`;
 - 释放内存块 `0`, `2`, `4`, 此时使用的内存块为 `[1, 3]`, 而空闲的内存块为 `[0, 2, 4]`;
 - 此时如果分配一个需要两个内存块的应用, 我们内存将分配失败。

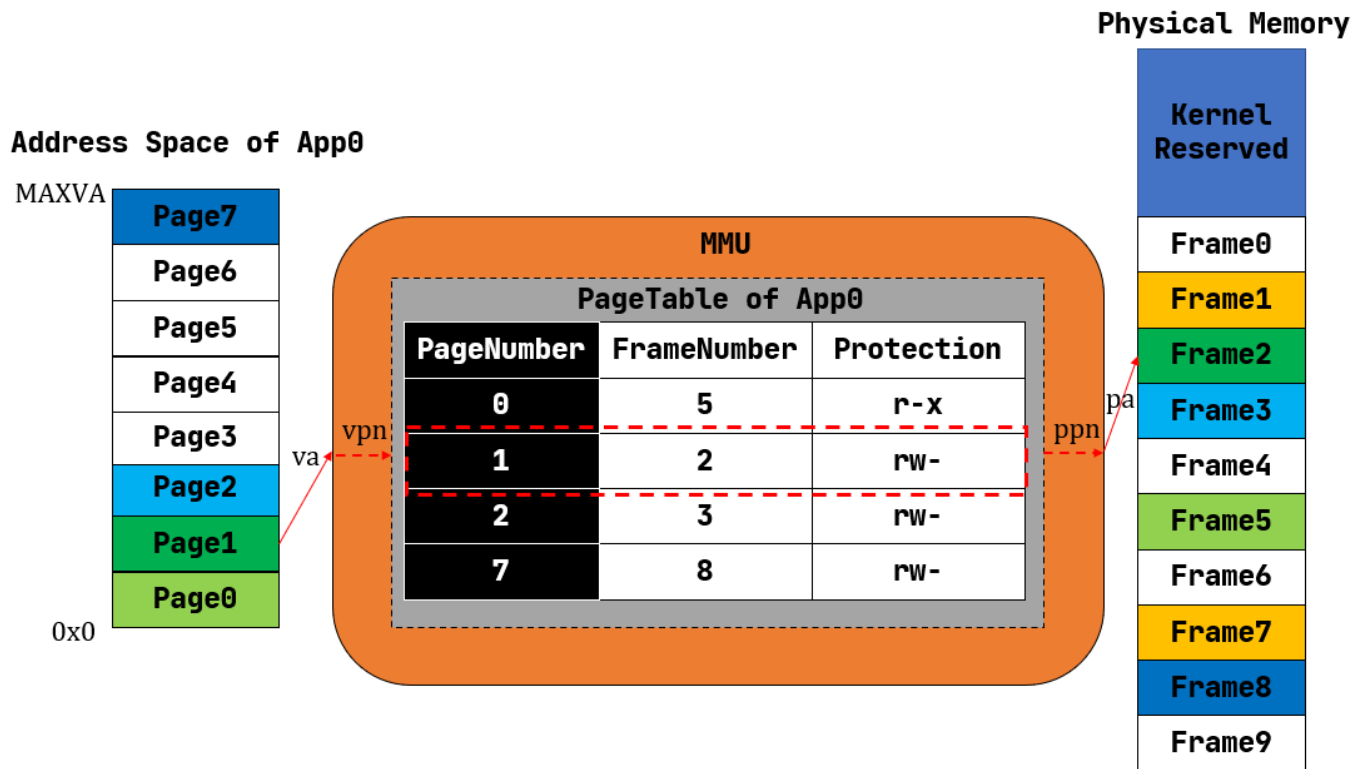
若要结合二者的优点的话, 就需要内核始终以一个**同样大小的单位**来在**物理内存**上放置应用地址空间中的数据:

1. 这样内核就可以使用简单的插槽式内存管理, 使得内存分配算法比较简单且不会产生外碎片;
2. 同时, 这个单位的大小要足够小, 从而其内部没有被用到的内碎片的大小也足够小, 尽可能提高内存利用率。

这便是我们将要介绍的分页内存管理, 总体的思想就是:

1. 假设我们每个代码块的大小是 `M`, 应用内存大小为 `AM`, 物理内存大小为 `PM`;
2. 那么应用可以分为 `AM / M` 个代码块, 物理内存可以分为 `PM / M` 个代码块;
3. 我们通过某些方式 (页表, MMU, TLB等), 将应用的代码块映射到物理内存的代码块;

带来的问题是, 我们需要引入更为复杂的硬件结构辅助我们映射应用内存到物理内存。



代码修改

